

目 录

1. 课程设计任务和目标	7
1.1. 任务目标	7
1.2. 项目分工	7
1.3. 开发环境	8
2. 需求分析	8
2.1. 数据流图	8
2.2. 源语言规范	9
2.2.1. 词法规范	9
2.2.2. 语法规则	9
2.2.3. 语义规范	10
2.3. 非功能性需求	11
3. 总体设计	12
3.1. 系统架构	12
3.2. 核心数据结构设计	13
3.2.1. AST 节点体系	13
3.2.2. 符号表	14
3.2.3. Token 元信息	15
3.2.4. Visitor 模式	16
3.3. 功能模块划分	16
3.3.1. 模块功能	16
3.3.2. 模块之间的关系	17
3.3.3. 各模块源文件及关系	18
3.4. 模块接口	19
3.4.1. 词法分析器接口	19
3.4.2. 语法分析器接口	19
3.4.3. 语义分析器接口	19
3.4.4. 代码生成器接口	20
3.4.5. 辅助工具接口	20
3.5. 用户接口设计	20
3.5.1. CLI	20

3.5.2.	TUI	20
3.5.3.	IDE	22
4.	详细设计	25
4.1.	词法分析模块	25
4.1.1.	输入输出	25
4.1.2.	数据结构	25
4.1.3.	关键算法	26
4.1.4.	关键规则片段	28
4.1.5.	错误处理	28
4.1.6.	BOM 处理	29
4.1.7.	文件结束处理	29
4.2.	语法分析模块	29
4.2.1.	输入输出	29
4.2.2.	数据结构	29
4.2.3.	关键算法	30
4.2.4.	错误处理	32
4.2.5.	语法规则覆盖	32
4.3.	AST 模块	33
4.3.1.	概述	33
4.3.2.	数据类型	33
4.3.3.	节点位置	33
4.3.4.	核心节点详解	34
4.3.5.	Arena 内存管理	36
4.3.6.	Visitor 模式	36
4.4.	符号表模块	36
4.4.1.	输入输出	36
4.4.2.	数据结构	37
4.4.3.	关键算法	38
4.4.4.	内置符号注册	40
4.5.	语义分析模块	40
4.5.1.	输入输出	40
4.5.2.	数据结构	40

4.5.3.	关键算法	41
4.5.4.	错误处理	48
4.6.	代码生成模块	48
4.6.1.	输入输出与总体架构	48
4.6.2.	四区代码缓冲	49
4.6.3.	类型映射	50
4.6.4.	变量声明生成	50
4.6.5.	常量声明生成	51
4.6.6.	过程与函数翻译	51
4.6.6.1.	原型声明	51
4.6.6.2.	过程定义	52
4.6.6.3.	函数定义与 <code>_retval</code> 机制	52
4.6.7.	<code>var</code> 引用参数的指针翻译	52
4.6.8.	数组访问的下标偏移	53
4.6.9.	表达式生成	54
4.6.9.1.	二元表达式	54
4.6.9.2.	一元表达式	54
4.6.9.3.	字面量与标识符	54
4.6.10.	控制流语句	54
4.6.10.1.	赋值语句	54
4.6.10.2.	<code>if-then-else</code> 语句	55
4.6.10.3.	<code>while</code> 语句	55
4.6.10.4.	<code>for</code> 循环	55
4.6.10.5.	<code>break</code> 与复合语句	55
4.6.11.	<code>read/write</code> 内建过程的翻译	56
4.6.12.	<code>Record</code> 类型的代码生成	56
4.6.13.	代码生成辅助函数	56
4.7.	错误处理模块	57
4.7.1.	输入输出	57
4.7.2.	数据结构	57
4.7.3.	错误恢复策略	57

4.8.	Record 类型扩展设计	57
4.8.1.	概述	57
4.8.2.	新增 AST 节点	58
4.8.3.	语法分析扩展	58
4.8.4.	字段访问标记机制	58
4.8.5.	语义分析扩展	58
4.8.6.	代码生成扩展	59
4.8.7.	Record 参数与数组支持	59
5.	源程序清单	61
5.1.	目录结构	61
5.2.	编程风格	61
6.	程序测试	63
6.1.	测试环境	63
6.2.	测试策略与架构	63
6.3.	测试用例总览	63
6.4.	词法分析测试	65
6.4.1.	词法单元测试	65
6.4.2.	词法合法用例 (valid/)	65
6.4.3.	词法非法用例 (invalid/)	66
6.5.	语法分析测试	67
6.5.1.	语法合法用例 (parser_valid/)	67
6.5.2.	语法非法用例 (parser_invalid/)	67
6.6.	语义分析测试	68
6.6.1.	语义分析 C++ 单元测试	68
6.6.2.	语义集成测试	70
6.6.2.1.	语义合法集成用例 (26 个)	71
6.6.2.2.	语义非法集成用例 (29 个)	72
6.7.	Record 类型专项测试	72
6.7.1.	Record 测试详细结果	74
6.8.	开放测试集 (Open Set)	74
6.8.1.	测试流程	75

6.8.2.	测试结果	75
6.8.3.	Open Set 用例清单	75
6.9.	测试脚本体系	77
6.10.	测试覆盖率分析	78
6.10.1.	词法阶段覆盖	78
6.10.2.	语法阶段覆盖	79
6.10.3.	语义阶段覆盖	79
6.10.4.	Record 扩展覆盖	79
6.10.5.	代码生成验证	80
6.11.	测试结果汇总	80
7.	课程设计总结	82
7.1.	项目完成情况	82
7.2.	每位成员的工作与收获	82
7.3.	设计过程中遇到的主要问题及解决方案	83
7.4.	改进建议	84
8.	附录 A: 完整文法	86
8.1.	程序结构	86
8.2.	列表定义	86
8.3.	常量声明	86
8.4.	类型声明	86
8.5.	变量声明	87
8.6.	子程序声明	87
8.7.	语句	88
8.8.	变量与过程调用	88
8.9.	表达式	89
8.10.	说明	89
9.	附录 B: 测试报告	90
9.1.	词法分析测试	90
9.1.1.	合法用例: 关键字大小写不敏感	90
9.1.2.	非法用例: 未终止字符字面量	91
9.2.	语法分析测试	92

9.2.1.	合法用例: <code>for-to</code> 循环的 AST 构建	92
9.2.2.	非法用例: <code>if</code> 缺少 <code>then</code>	92
9.3.	语义分析测试	93
9.3.1.	合法用例: 阶乘递归函数	93
9.3.2.	合法用例: 多维数组	94
9.3.3.	非法用例: 类型不匹配赋值	94
9.3.4.	非法用例: 数组下标越界	95
9.4.	Record 专项测试	96
9.4.1.	合法用例: <code>record</code> 数组求总年龄	96
9.4.2.	合法用例: <code>record</code> 作为 <code>var</code> 参数	97
9.4.3.	非法用例: 访问不存在的 <code>record</code> 字段	99
9.5.	课件示例测试	100
9.6.	Open Set 集成测试	100
9.6.1.	扩展欧几里得算法	100
9.6.2.	图着色计数 (DP)	102
10.	附录 C: Git 日志	103
11.	附录 D: 大模型使用情况记录	111

Pascal-S 语言编译程序的设计与实现

1. 课程设计任务和目标

本项目是编译原理与技术课程的课程设计，目标是设计并实现一个完整的编译程序，将 Pascal-S（Pascal 语言的一个教学子集）源代码翻译为 C 语言目标代码。编译器采用经典的编译前端架构，依次完成词法分析、语法分析、语义分析和代码生成四个阶段。

1.1 任务目标

- 实现完整的 Pascal-S 到 C 语言的编译流程，生成可编译运行的 C 代码。
- 采用 Flex 实现词法分析器，Bison 实现 LALR(1) 语法分析器。
- 在语法分析阶段构建抽象语法树（AST），语义分析阶段对 AST 进行类型推导和注解，代码生成阶段基于 Visitor 模式遍历注解后的 AST 生成目标代码。
- 支持 Pascal-S 的核心语法特性：基本类型、数组、过程/函数、参数传递（值传递与引用传递）、控制流语句。
- 扩展支持 record 类型（结构体），包括 record 变量的字段访问、record 作为参数、record 数组等。

1.2 项目分工

本项目由五名成员协作完成，分工如下：

角色	姓名	职责
语法分析	王嘉晗	AST 契约定义、Bison 建树动作、解析结果导出、单元测试
组长/语义分析	张宸宇	统筹架构、模块联调、集成测试；符号表实现、AST 语义注解、类型检查、语义错误处理、单元测试
词法分析	胡航宾	Flex 词法规则、Token 产出、行列号维护、词法错误检测、单元测试
代码生成（基础）	李思远	Visitor 基础遍历、基础语句生成、代码生成框架
代码生成（进阶）	谢康	数组访问、过程调用、传引用参数、复杂节点代码生成

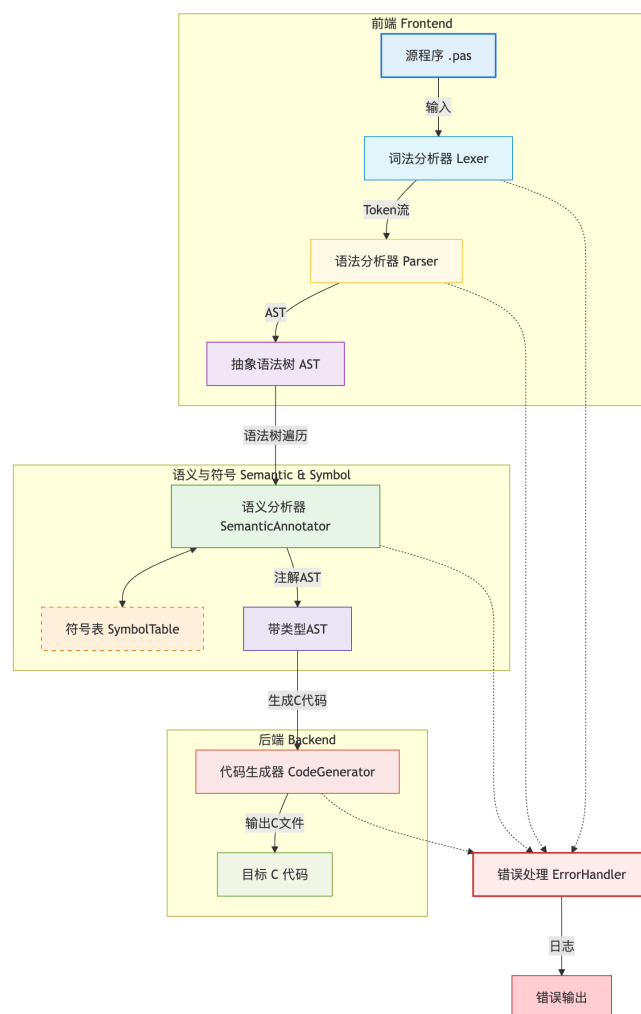
1.3 开发环境

- 操作系统: macOS / Windows / Linux
- 编译器: g++ (C++17 标准)
- 词法/语法工具: Flex 2.6.4 / Bison 3.8.2
- 构建工具: GNU Make / CMake
- 版本控制: Git
- 代码统计工具: cloc

2. 需求分析

2.1 数据流图

编译器整体数据流如下图所示: Pascal-S 源文件输入后, 依次经过词法分析、语法分析、语义分析、代码生成四个阶段, 最终输出 C 语言源文件。



编译器数据流图

2.2 源语言规范

2.2.1 词法规范

Pascal-S 的词法规则遵循标准 Pascal 规范，关键特征如下：

1. **标识符**：以字母开头，后跟字母或数字。不区分大小写，词法阶段统一转换为小写。
2. **关键字**：共 33 个保留字，包括 `program`、`begin`、`end`、`const`、`var`、`type`、`record`、`procedure`、`function`、`if`、`then`、`else`、`while`、`do`、`for`、`to`、`downto`、`break`、`read`、`write`、`array`、`of`、`integer`、`real`、`boolean`、`char`、`true`、`false`、`not`、`and`、`or`、`div`、`mod`。
3. **常量**：支持整数、实数（含科学计数法）、十六进制数（`0x` 和 `$` 前缀）、字符常量（单引号）、字符串常量。
4. **运算符**：赋值 `:=`，关系比较（`=`、`<>`、`<`、`>`、`<=`、`>=`），算术运算（`+`、`-`、`*`、`/`、`div`、`mod`），逻辑运算（`and`、`or`、`not`）。
5. **注释**：支持 `{ }` 风格的注释，且支持嵌套注释。注释可出现在任何单词之后。
6. **编译指令**：支持 `{$....}` 形式的编译器指令（如 `{IFDEF}`、`{ENDIF}`）。

2.2.2 语法规范

编译器采用 LALR(1) 分析方法，接受完整的 Pascal-S 文法。程序主体结构如下：

- **程序结构**：`program id (idlist) ; program_body .`——程序头后跟常量声明、类型声明、变量声明、子程序声明和复合语句。
- **声明顺序**：常量声明 `const ...` → 类型声明 `type ...` → 变量声明 `var ...` → 子程序声明 `procedure/function ...`。
- **类型系统**：支持 `integer`、`real`、`boolean`、`char` 四种基本类型，以及 `array [lower..upper] of type` 数组类型，和扩展的 `record ... end` 记录类型。
- **子程序**：支持过程和函数定义，参数支持值传递和引用传递（`var` 参数）。过程和函数不允许嵌套定义。
- **语句**：支持赋值、过程调用、复合语句（`begin...end`）、条件语句（`if-then-else`）、`for` 循环（`to/downto`）、`while` 循环、`break` 语句。
- **表达式**：支持算术、关系、逻辑运算，以及函数调用表达式。

- **变量访问：**支持标识符、数组下标 (`a[i]`)、字段访问 (`r.field`)。

2.2.3 语义规范

编译器在语义分析阶段实施以下检查：

1. 类型系统：

- `integer` 映射为 C 的 `int`
- `real` 映射为 C 的 `float`
- `boolean` 映射为 C 的 `int` (0 表示 `false`，非 0 表示 `true`)
- `char` 映射为 C 的 `char`
- `record` 映射为 C 的 `typedef struct`

2. 作用域规则：

采用两层作用域（全局作用域 + 局部作用域）。过程/函数由于不可嵌套定义，局部作用域最多一层。内层可访问外层变量，同名局部变量屏蔽外层变量。

3. 参数传递：

- 值传递：复制实参值，C 实现为普通参数。
- 引用传递 (`var`)：传递地址，C 实现为指针参数，调用时传地址 `&arg`，使用需解引用 `*arg`。

4. 数组处理：

Pascal-S 数组可以从任意整数开始（如 `array[3..9] of integer`），而 C 数组从 0 开始，编译器需自动计算下标偏移： $a[i] \rightarrow a[i - \text{lower}]$ 。

5. 类型检查：

- 变量/函数使用前必须声明。
- 同一作用域内名字不可重复定义。
- 赋值语句左右类型兼容（允许 `integer` $\rightarrow$ `real` 隐式转换）。
- 数组下标表达式必须为 `integer` 类型。
- 过程/函数调用参数个数和类型必须匹配。
- `break` 语句必须在循环体内。
- 函数结果赋值 (`func_name := expr`) 只允许出现在对应函数体内。

6. Record 类型扩展：

- 支持类型声明 `type T = record ... end;`
- 支持字段访问 `rec_var.field_name`

- 字段访问可作为赋值左值
- 支持 record 作为函数/过程参数
- 支持 record 数组

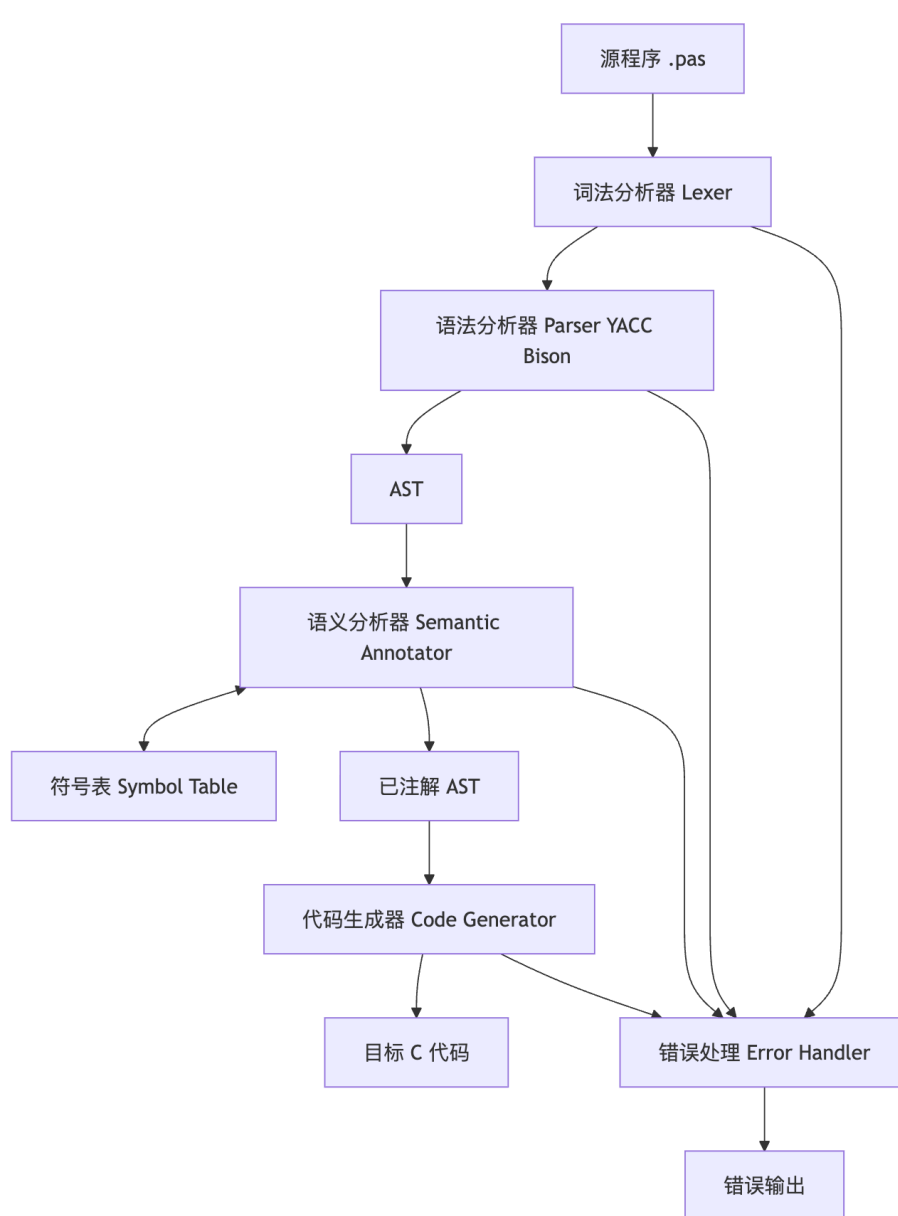
2.3 非功能性需求

- 编译速度：千行以内 Pascal-S 代码应在数秒内完成编译。
- 内存管理：采用 Arena 模式管理 AST 节点和符号表条目生命周期，避免内存泄漏。
- 错误恢复：各阶段遇到错误不应立即终止，应尽可能多地收集并报告错误。
语法分析采用恐慌模式在分号处恢复。
- 可扩展性：Visitor 模式使代码生成逻辑与 AST 结构解耦，便于新增节点类型和语义检查。

3. 总体设计

3.1 系统架构

编译器采用经典的“词法分析 → 语法分析 → 语义分析 → 代码生成”四阶段流水线架构。各阶段通过中间表示进行数据传递：



编译器总体结构图

编译流程如下：

1. Pascal-S 源文件（.pas）首先由 Flex 词法分析器进行词法分析，输出 Token 流。

2. Token 流由 Bison 语法分析器消费，进行 LALR(1) 分析，在归约过程中调用 ASTBuilder 构建抽象语法树。
3. 语法分析输出的 AST 根节点（ProgramNode）传递给语义分析器。
4. 语义分析器递归遍历 AST，进行类型推导、符号绑定和各类语义检查，为 AST 节点注解 dataType 和 symbolEntry。
5. 代码生成器基于 Visitor 模式遍历注解后的 AST，分四个缓冲区收集代码，最终由 CodegenUtils::wrapAsCProgram 组装为完整 C 程序输出。

3.2 核心数据结构设计

3.2.1 AST 节点体系

抽象语法树是整个编译器的核心数据结构，定义了统一的 ASTNode 基类及 25 种派生节点类型：

节点类	NodeType	对应文法结构
ProgramNode	Program	程序根节点，包含程序名和 Body
BlockNode	Block	作用域块，子节点：consts/types/vars/subs/compound
VarDeclNode	VarDecl	变量声明：idlist : type
ConstDeclNode	ConstDecl	常量声明：id = const_value
ProcDeclNode	ProcDecl	过程声明：procedure id(params); body
FuncDeclNode	FuncDecl	函数声明：function id(params): type; body
AssignStmtNode	AssignStmt	赋值语句：var := expr
IfStmtNode	IfStmt	条件语句：if expr then stmt else stmt
WhileStmtNode	WhileStmt	while 循环：while expr do stmt
ForStmtNode	ForStmt	for 循环：for id := expr to/downto expr do stmt
BreakStmtNode	BreakStmt	break 语句
CompoundStmtNode	CompoundStmt	复合语句：begin stmt_list end
ProcCallNode	ProcCall	过程/函数调用：id(args)
BinaryExprNode	BinaryExpr	二元表达式：expr op expr
UnaryExprNode	UnaryExpr	一元表达式：op expr
IdentifierNode	Identifier	标识符引用
LiteralNode	Literal	字面量

ArrayAccessNode	ArrayAccess	数组下标访问: arr[index]
ArrayTypeNode	ArrayType	数组类型定义: array[lower..upper] of type
ParamDeclNode	ParamDecl	参数声明: var idlist : type
TypeDeclNode	TypeDecl	类型声明: type name = typedef
RecordTypeNode	RecordType	record 类型定义: record fields end
FieldDeclNode	FieldDecl	字段声明: idlist : type
FieldAccessNode	FieldAccess	字段访问: base.field
ListNode	List	通用列表容器, ListKind 区分标识符/语句/表达式/参数等列表

ASTNode 基类包含以下核心字段:

- **nodeType**: 节点类型枚举值 (NodeType)。
- **dataType**: 推导出的数据类型 (DataType), 初始值为 Unknown, 语义分析阶段填充。
- **pos**: 源码位置 (SourcePos), 包含行号和列号。
- **children**: 子节点指针的向量 (std::vector<ASTNode*>)。
- **symbolEntry**: 符号表条目指针, 语义分析阶段绑定。

ListNode 是特殊的通用容器节点, 通过 ListKind 枚举区分用途:

Identifiers (标识符列表)、Statements (语句列表)、Expressions (表达式列表)、Parameters (参数列表)、Declarations (声明列表)、ArrayRanges (多维数组范围)、FieldAccess (字段访问链)。

AST 对象通过 ASTBuilder 类进行统一创建和管理。ASTBuilder 内部维护一个 std::vector<std::unique_ptr<ASTNode>> Arena, 所有节点通过模板方法 create<T>(…) 创建后追加到 Arena 中。这种设计确保了:

- 指针稳定性: 所有节点在 Arena 生命周期内地址不变。
- 内存安全: Arena 析构时自动释放所有节点。
- 使用便利性: 调用者无需关心内存管理。

3.2.2 符号表

符号表采用“哈希表 + 栈”结构实现, 用于管理编译过程中的符号信息:

- 底层数据结构: std::vector<std::unordered_map<std::string, const SymbolEntry*>>

- 每个作用域对应一个哈希表（`unordered_map`）。
- 作用域以栈方式管理：`enterScope()` 压入新表，`exitScope()` 弹出。
- 符号名 `Key` 统一经 `normalizeName()` 转为小写。
- 插入时写入当前作用域，查找（`lookup()`）从当前作用域向全局方向遍历。

符号条目 `SymbolEntry` 包含以下字段：

字段	类型	说明
<code>name</code>	<code>string</code>	符号名称（小写）
<code>kind</code>	<code>SymbolKind</code>	符号种类：Variable/Constant/ Procedure/Function/Parameter/ TypeAlias
<code>type</code>	<code>DataType</code>	数据类型：Integer/Real/Boolean/Char/ Record 等
<code>scopeLevel</code>	<code>int</code>	作用域层级（0=全局，1=局部）
<code>isArray</code>	<code>bool</code>	是否为数组类型
<code>arrayBounds</code>	<code>vector</code>	多维数组各维的上下界
<code>hasConstLiteral</code>	<code>bool</code>	是否是常量字面量
<code>constLiteralText</code>	<code>string</code>	常量字面量文本
<code>isStringLikeConst</code>	<code>bool</code>	是否为类字符串常量
<code>params</code>	<code>vector</code>	过程/函数的参数列表
<code>isVarParam</code>	<code>bool</code>	是否为 <code>var</code> 引用参数
<code>fields</code>	<code>vector</code>	Record 类型的字段列表
<code>typeName</code>	<code>string</code>	用户自定义类型的名称

`SymbolEntry` 使用静态工厂方法创建：`makeVariable()`、`makeConstant()`、`makeProcedure()`、`makeFunction()`、`makeParameter()`、`makeTypeAlias()`。

符号表的 `entryArena_`（`std::vector<std::unique_ptr<SymbolEntry>>`）确保符号条目生命周期与符号表一致，即使作用域退出后指针仍然有效，便于代码生成阶段回溯引用。

3.2.3 Token 元信息

项目中定义了 `include/token.h` 作为统一的 Token 描述结构，用于表达 `type/lexeme/line/column` 四类基础信息；在主编译链路中，词法阶段并不是以 Token 对象数组的形式向前传递数据，而是由 Flex 直接返回 Bison Token 编号，并通过 `yylval`、`yylloc` 以及词法模块查询函数暴露 Token 元信息。字段如下：

字段	位置	说明
token kind	yyllex() 返回值	关键字、标识符、数字、字符、字符串、多字符运算符或单字符 ASCII Token
lexeme	lexerLastLexeme() / yytext	最近一次识别到的词素文本
type label	lexerLastType()	用于 --lex / --dump-tokens 输出的分类标签，如 Keyword、Identifier、Number
rule label	lexerLastRule()	规则命中标签，如 keyword_identifier、real_number、assign_operator
line / column	yylloc / lexerLastLine() / lexerLastColumn()	Token 起始行列号，采用 1-based 计数

3.2.4 Visitor 模式

编译器采用 Visitor 模式实现 AST 遍历逻辑的分离。ASTVisitor 定义纯虚接口，为每种节点类型提供 visit() 方法。SemanticAnnotator 和 CodeGenerator 均实现该接口，分别实现语义注解和代码生成两个正交关注点。

3.3 功能模块划分

3.3.1 模块功能

各功能模块的职责如下：

1. 词法分析模块 (Lexer)：

- 使用 Flex 实现，定义在 src/lexer.l。
- 识别所有 Token 类型并返回给 Bison 语法分析器。
- 标识符统一转换为小写。
- 跟踪行号和列号，为每个 Token 提供精确的位置信息。
- 检测并报告词法错误（非法字符、未闭合注释、数值溢出等）。
- 支持嵌套注释和编译器指令。

2. 语法分析模块 (Parser)：

- 使用 Bison 实现，定义在 src/parser.y。

- 实现 LALR(1) 分析，接受 Pascal-S 完整文法。
- 在归约动作中调用 ASTBuilder 构建 AST 节点。
- 遇到语法错误后通过恐慌模式恢复（丢弃 Token 直到遇到分号）。
- 输出 ProgramNode* 根节点供后续阶段消费。

3. 语义分析模块 (Semantic Annotator) :

- 定义在 src/semantic_annotator.cpp，是最大的源文件（约 1250 行）。
- 递归遍历 AST，为每个节点注解类型信息和符号表引用。
- 实现完整的类型检查、作用域检查、参数验证。
- 支持编译期常量表达式求值（用于数组边界检查）。
- 上下文跟踪：函数结果赋值检测、循环深度跟踪（用于 break 验证）。

4. 代码生成模块 (Code Generator) :

- 定义在 src/code_generator.cpp（约 630 行）和 src/codegen_utils.cpp（约 440 行）。
- 基于 Visitor 模式遍历注解后的 AST。
- 将代码分为四个缓冲区收集：全局声明、函数原型、函数定义、main 函数体。
- 最终由 wrapAsCProgram() 组装为完整的 C 源文件。

5. 错误处理模块 (Error Handler) :

- 定义在 include/error_handler.h 和 src/error_handler.cpp。
- 提供统一的错误报告接口：report(line, col, message)。
- 存储 std::vector<CompileError>，记录所有编译错误。
- 词法错误通过词法分析器内部独立的错误向量管理。

3.3.2 模块之间的关系

各模块之间通过明确的接口进行解耦协作：

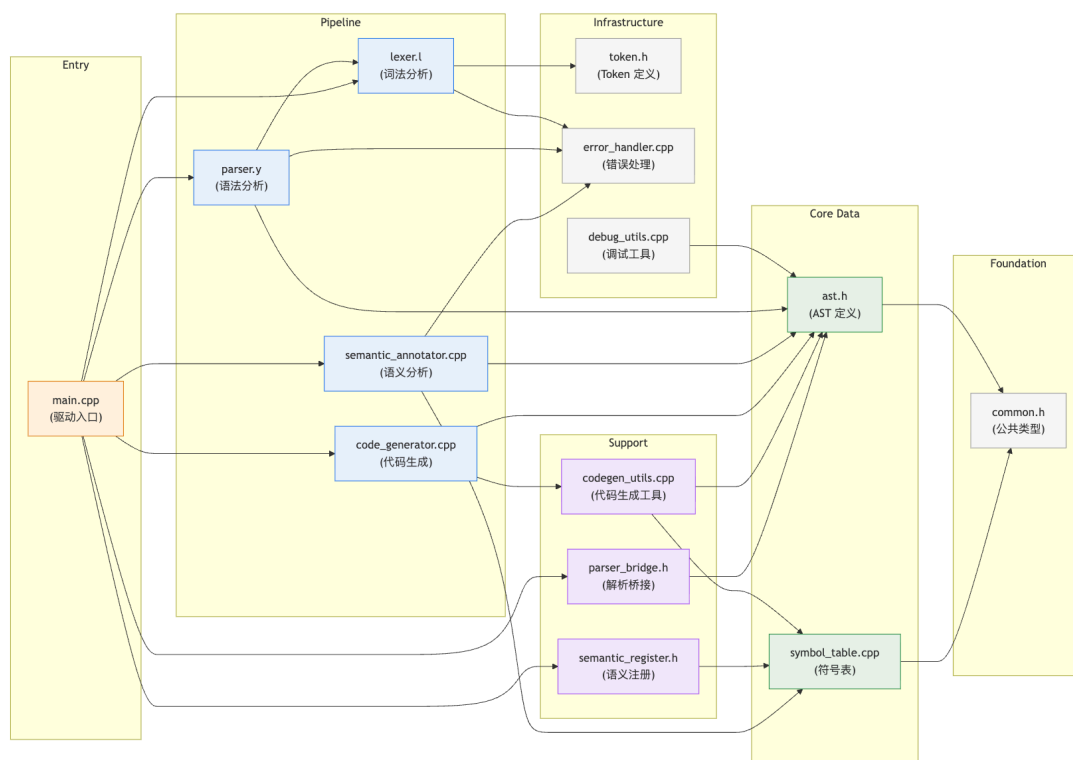
- 词法分析 → 语法分析：Flex 提供的 yylex() 每次返回一个 Token，Bison 通过 Bison 内置的 yylex() 接口消费 Token 流。
- 语法分析 → 语义分析：语法分析结束后输出 ProgramNode* 根节点，语义分析器以该节点为入口遍历 AST。
- 语义分析 → 代码生成：语义分析器为 AST 节点填充 dataType 和 symbolEntry，代码生成器读取这些注解信息生成代码。

- **所有模块 → 错误处理**：各模块均可通过统一的 `ErrorHandler::report()` 接口报告错误。

3.3.3 各模块源文件及关系

模块	实现文件	功能说明
词法分析器	src/lexer.l	Flex 实现，负责 Token 识别、标识符大小写归一化、注释处理、行列号维护、词法错误检测
语法分析器	src/parser.y	Bison 实现，LALR(1) 分析，归约时构建 AST，恐慌模式错误恢复
AST 数据结构	include/ast.h, src/ast.cpp	20 种 AST 节点类型，Visitor 模式访问接口，Arena 内存管理
符号表	include/symbol_table.h, src/symbol_table.cpp	哈希表+栈结构，两层作用域管理，支持变量/常量/过程/函数/类型别名
语义分析器	include/semantic_annotator.h, src/semantic_annotator.cpp	递归遍历 AST，类型推导与检查，符号绑定，语义错误报告
代码生成器	include/code_generator.h, src/code_generator.cpp	Visitor 模式遍历注解后 AST，分四区收集代码，生成完整 C 程序
代码生成工具	include/codegen_utils.h, src/codegen_utils.cpp	类型映射、声明生成、读写语句、代码组装等辅助函数
错误处理	include/error_handler.h, src/error_handler.cpp	统一错误报告接口，记录行号列号，错误恢复策略
主程序	src/main.cpp	命令行解析，编译流程控制，事件流输出，调试支持

程序源文件之间的依赖关系如下图：



3.4 模块接口

3.4.1 词法分析器接口

词法分析器导出以下 C 函数供外部调用：

```

01 void lexerResetState();           // 重置词法分析器状态
02 const char* lexerLastLexeme();    // 返回最后一个词素
03 const char* lexerLastType();      // 返回最后一个 Token 的类型
04 const char* lexerLastRule();      // 返回最后一次命中的规则标签
05 int lexerLastLine();              // 返回最后一个 Token 的行号
06 int lexerLastColumn();            // 返回最后一个 Token 的列号
07 void lexerClearErrors();          // 清空错误列表
08 int lexerErrorCount();             // 返回词法错误计数
09 const CompileError* lexerErrorAt(int index); // 获取指定词法错误
10 int lexerAllocatedStringCount();  // 返回 strdup 次数
11 int lexerTokenCount();            // 返回 Token 计数
  
```

3.4.2 语法分析器接口

```

01 int yyparse();                   // Bison 生成的分析函数
02 ProgramNode* getParseResultRoot(); // 返回解析结果的根节点
03 void resetParseResult();          // 重置解析结果
04 int getParseErrorCount();          // 返回语法错误计数
  
```

3.4.3 语义分析器接口

```
01 void SemanticAnnotator::annotate(ASTNode* root); // 遍历 AST 执行语义注解
```

3.4.4 代码生成器接口

```
01 std::string CodeGenerator::generate(ProgramNode* root); // 从 AST 根节点生成 C 代码
```

3.4.5 辅助工具接口

```
01 std::string CodegenUtils::mapType(DataType t); // 类型映射
02 std::string CodegenUtils::wrapAsCProgram(globals, prototypes, definitions,
mainBody); // 组装 C 程序
03 std::string CodegenUtils::emitVarDecl(VarDeclNode*); // 变量声明生成
04 std::string CodegenUtils::emitConstDecl(ConstDeclNode*); // 常量声明生成
05 std::string CodegenUtils::emitProcPrototype(ProcDeclNode*); // 过程原型生成
06 std::string CodegenUtils::emitFuncPrototype(FuncDeclNode*); // 函数原型生成
07 std::string CodegenUtils::emitProcDecl(ProcDeclNode*); // 过程定义生成
08 std::string CodegenUtils::emitFuncDecl(FuncDeclNode*); // 函数定义生成
09 std::string CodegenUtils::emitReadStmt(ASTNode* node); // read 语句生成
10 std::string CodegenUtils::emitWriteStmt(ASTNode* node); // write 语句生成
```

3.5 用户接口设计

3.5.1 CLI

编译器以命令行工具形式运行，支持以下参数：

```
zcy ~ % pasc
pasc - Pascal-S to C compiler (project skeleton)
Usage: pasc -i <input.pas> [-o output.c] [--lex] [--dump-tokens] [--parse] [--semantic] [--dump-annotated-ast]
运行会截止到参数所指步骤
```

```
01 pasc -i <input.pas> [-o output.c] [选项]
02 选项:
03     --lex          仅执行词法分析，输出 Token 列表
04     --dump-tokens  词法分析并输出 Token 详情及规则命中信息
05     --parse       执行词法和语法分析后停止，输出 AST
06     --semantic    执行词法、语法和语义分析后停止
07     --dump-annotated-ast 语义分析后输出带注解的 AST
08     --help        显示帮助信息
```

若未指定 `-o` 参数，编译器自动将输出文件名设为与输入文件同名但扩展名为 `.c` 的文件。

3.5.2 TUI

编译器还支持通过环境变量 `PASC_EVENT_STREAM=1` 启用结构化 JSON 事件流输出，供 TUI（终端用户界面）等工具实时监控编译进度。日志详尽程度由环境变量 `PASC_LOG_LEVEL` 控制（支持 `debug`、`info`、`warn`、`error` 级别）。

作为本课设的亮点之一，我们基于 Python 设计了终端用户界面，直观展示编译过程的主要信息。

在主界面选择文件、参数等：



```
zcy — Python • pascct — 100x30

— PASCCT 编译器交互式界面 —
Pascal-S 到 C 的编译器

使用 ↑↓ 导航, ←→ 调整选项, Enter 选择/编辑, q 退出

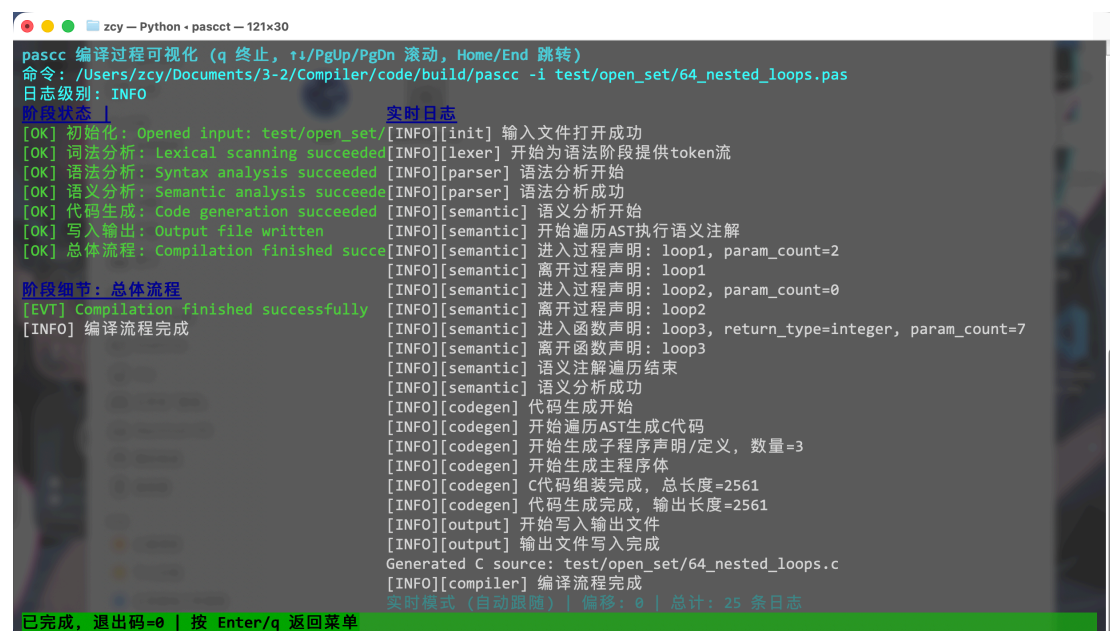
► 输入文件      : 未选择
  输出文件      : 自动生成
  编译器选项    : 无
  日志级别      : INFO
  额外参数      : 无

  开始编译
  退出

可用的 Pascal 文件 (按 Enter 进入浏览模式):
1. test/cases/invalid/i01_illegal_character_at.pas
2. test/cases/invalid/i02_illegal_character_hash.pas
3. test/cases/invalid/i03_malformed_number_multiple_dots.pas
4. test/cases/invalid/i04_malformed_number_incomplete_exponent.pas
5. test/cases/invalid/i05_terminated_char_literal.pas
6. test/cases/invalid/i06_char_literal_too_long.pas
7. test/cases/invalid/i07_terminated_comment_brace.pas
8. test/cases/invalid/i08_terminated_string_literal.pas
9. test/cases/invalid/i09_terminated_directive.pas
10. test/cases/invalid/i10_malformed_hex_number.pas
(显示 1-10/251)

提示: 在输入文件上按 Enter 进入浏览模式 | q 退出
```

运行后，显示编译过程中 INFO 级别的日志：



```
zcy — Python • pascct — 121x30

pascct 编译过程可视化 (q 终止, ↑↓/PgUp/PgDn 滚动, Home/End 跳转)
命令: /Users/zcy/Documents/3-2/Compiler/code/build/pascct -i test/open_set/64_nested_loops.pas
日志级别: INFO

阶段状态 | 实时日志
[OK] 初始化: Opened input: test/open_set/64_nested_loops.pas [INFO][init] 输入文件打开成功
[OK] 词法分析: Lexical scanning succeeded [INFO][lexer] 开始为语法阶段提供token流
[OK] 语法分析: Syntax analysis succeeded [INFO][parser] 语法分析开始
[OK] 语义分析: Semantic analysis succeeded [INFO][parser] 语法分析成功
[OK] 代码生成: Code generation succeeded [INFO][semantic] 语义分析开始
[OK] 写入输出: Output file written [INFO][semantic] 开始遍历AST执行语义注解
[OK] 总体流程: Compilation finished successfully [INFO][semantic] 进入过程声明: loop1, param_count=2
[INFO] 编译流程完成 [INFO][semantic] 离开过程声明: loop1
[INFO][semantic] 进入过程声明: loop2, param_count=0
[INFO][semantic] 离开过程声明: loop2
[INFO][semantic] 进入函数声明: loop3, return_type=integer, param_count=7
[INFO][semantic] 离开函数声明: loop3
[INFO][semantic] 语义注解遍历结束
[INFO][semantic] 语义分析成功
[INFO][codegen] 代码生成开始
[INFO][codegen] 开始遍历AST生成C代码
[INFO][codegen] 开始生成子程序声明/定义, 数量=3
[INFO][codegen] 开始生成主程序体
[INFO][codegen] C代码组装完成, 总长度=2561
[INFO][codegen] 代码生成完成, 输出长度=2561
[INFO][output] 开始写入输出文件
[INFO][output] 输出文件写入完成
Generated C source: test/open_set/64_nested_loops.c
[INFO][compiler] 编译流程完成
实时模式 (自动跟踪) | 偏移: 0 | 总计: 25 条日志

已完成, 退出码=0 | 按 Enter/q 返回菜单
```

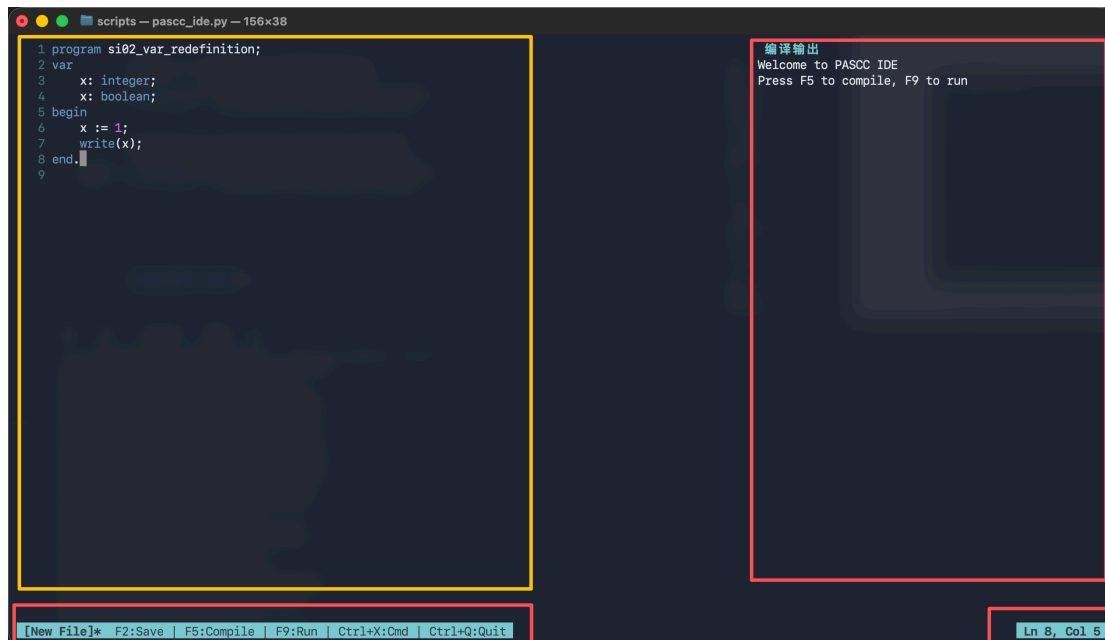
也支持显示更详细的 DEBUG 级别日志，包括变量注册等信息：

```
zcy - Python - pascc - 121x30
pascc 编译过程可视化 (q 终止, ↑/PgUp/PgDn 滚动, Home/End 跳转)
命令: /Users/zcy/Documents/3-2/Compiler/code/build/pascc -i test/open_set/64_nested_loops.pas
日志级别: DEBUG
阶段状态 \ 实时日志
[OK] 初始化: Opened input: test/open_set/ [DEBUG][semantic] 注册变量: f type=integer
[OK] 词法分析: Lexical scanning succeeded [DEBUG][semantic] 注册变量: g type=integer
[OK] 语法分析: Syntax analysis succeeded [INFO][semantic] 离开过程声明: loop2
[OK] 语义分析: Semantic analysis succeeded [INFO][semantic] 进入函数声明: loop3, return_type=integer, param_count=7
[OK] 代码生成: Code generation succeeded [DEBUG][semantic] 注册参数: h type=integer
[OK] 写入输出: Output file written [DEBUG][semantic] 注册参数: i type=integer
[OK] 总体流程: Compilation finished succe [DEBUG][semantic] 注册参数: j type=integer
[DEBUG][semantic] 注册参数: k type=integer
[DEBUG][semantic] 注册参数: l type=integer
[DEBUG][semantic] 注册参数: m type=integer
[DEBUG][semantic] 注册参数: n type=integer
[DEBUG][semantic] 注册变量: ans type=integer
[DEBUG][semantic] 注册变量: a type=integer
[DEBUG][semantic] 注册变量: b type=integer
[DEBUG][semantic] 注册变量: c type=integer
[DEBUG][semantic] 注册变量: d type=integer
[DEBUG][semantic] 注册变量: e type=integer
[DEBUG][semantic] 注册变量: f type=integer
[DEBUG][semantic] 注册变量: g type=integer
[INFO][semantic] 离开函数声明: loop3
[DEBUG][semantic] 分析调用: loop1, arg_count=2
[DEBUG][semantic] 分析调用: loop2, arg_count=0
[DEBUG][semantic] 分析调用: loop3, arg_count=7
[DEBUG][semantic] 分析调用: write, arg_count=1
滚动模式 ↑ | 偏移: 19 | 总计: 83 条日志
已完成, 退出码=0 | 按 Enter/q 返回菜单
```

3.5.3 IDE

在终端用户界面的基础上，我们还设计了支持 Pascal-S 的终端 IDE。用户可在此用 Pascal-S 编程，可调用 Pascc 直接编译出 C 语言文件，并可进一步将 C 程序编译成可执行文件并运行。

IDE 界面如下图，黄色区域为代码编辑区，支持语法高亮；下方为功能键提示；右侧为输出区；右下角提示光标行列号：



F2 保存，F5 编译，此时编译失败（符号 x 重定义），支持错误高亮：

```
1 program si02_var_redefinition;
2 var
3   x: integer;
4   x: real;
5 begin
6   x := 1;
7   write(x);
8 end.
9
```

编译输出
== Compiling ==
File: /Users/zcy/Documents/3-2/Compiler/code/s...
Error at 4:5 - Redefinition of identifier: x
Error at line 4: Redefinition of identifier: x
[FAILED] Compilation failed

2.pas Compilation failed Ln 4, Col 1

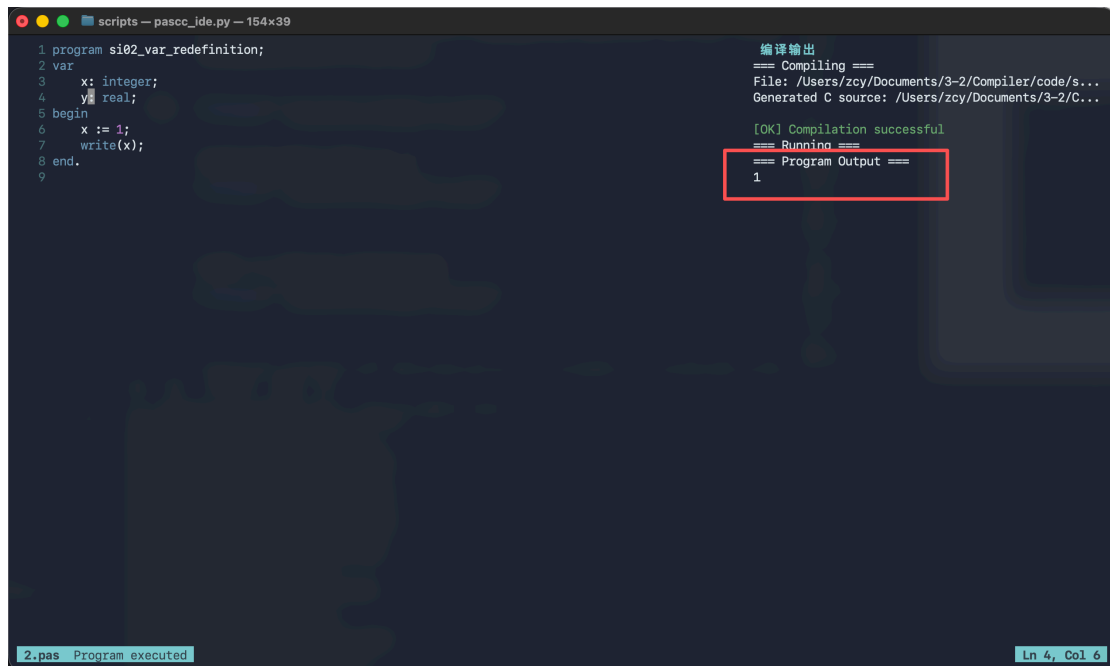
将第二个 x 修改成 y 后，重新保存并编译，显示编译成功：

```
1 program si02_var_redefinition;
2 var
3   x: integer;
4   y: real;
5 begin
6   x := 1;
7   write(x);
8 end.
9
```

编译输出
== Compiling ==
File: /Users/zcy/Documents/3-2/Compiler/code/s...
Generated C source: /Users/zcy/Documents/3-2/C...
[OK] Compilation successful

2.pas Compilation successful Ln 4, Col 6

按 F9 执行，右侧显示代码运行结果：



```
1 program si02_var_redefinition;
2 var
3   x: integer;
4   y: real;
5 begin
6   x := 1;
7   write(x);
8 end.
9
```

编译输出
=== Compiling ===
File: /Users/zcy/Documents/3-2/Compiler/code/s...
Generated C source: /Users/zcy/Documents/3-2/C...
[OK] Compilation successful
=== Running ===
=== Program Output ===
1

2.pas Program executed Ln 4, Col 6

```
01 def run_compiled_file(c_file: str, project_root: str) -> str:
02     """用gcc编译Pascc输出的C语言文件并运行"""
03     exe_file = c_file.rsplit(".", 1)[0]
04
05     # 用gcc编译, 实质上是终端命令
06     gcc_cmd = ["gcc", "-o", exe_file, c_file, "-lm"]
07     try:
08         result = subprocess.run(gcc_cmd, capture_output=True, text=True,
09                                 timeout=10)
10         if result.returncode != 0:
11             return f"GCC compilation failed:\n{result.stderr}"
12     except Exception as e:
13         return f"GCC error: {str(e)}" # 显示gcc报错
14
15     # 执行程序
16     try:
17         result = subprocess.run([exe_file], capture_output=True, text=True,
18                                 timeout=5)
19         return f"=== Program Output ===\n{result.stdout}\n{result.stderr}"
20     except subprocess.TimeoutExpired: # 运行超时检测
21         return "Program execution timeout"
22     except Exception as e:
23         return f"Execution error: {str(e)}"
```


4. 详细设计

4.1 词法分析模块

4.1.1 输入输出

- 输入：Pascal-S 源文件（.pas）
- 输出：Token 流（由 yylex() 每次返回一个 Token，供 Bison 语法分析器消费）

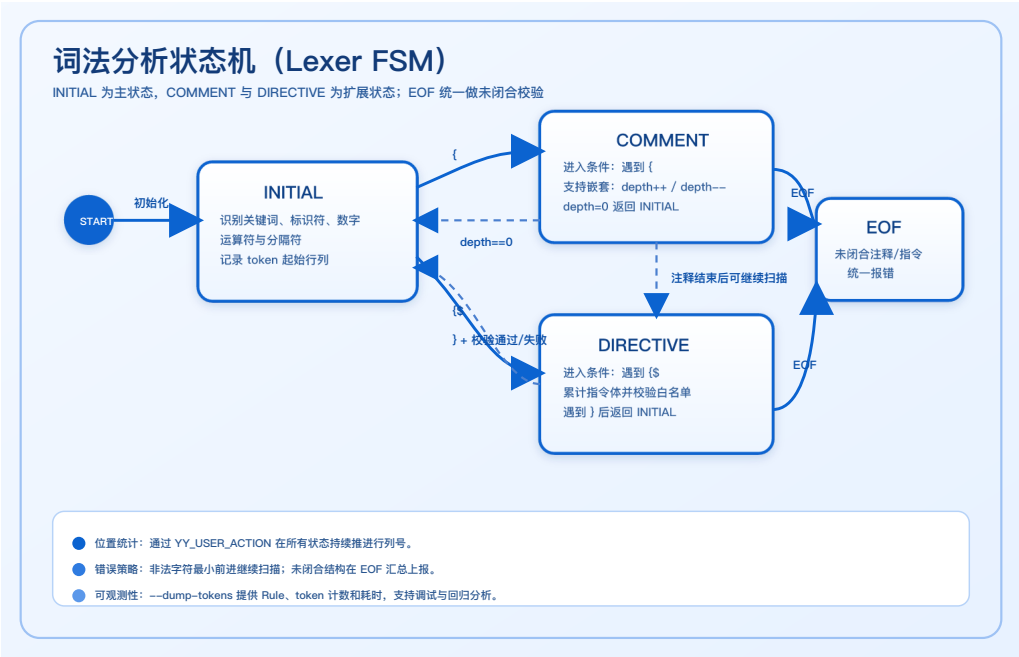
4.1.2 数据结构

词法分析器使用全局状态变量管理扫描上下文：

```
01 int g_line = 1;           // 当前行号
02 int g_column = 1;        // 当前列号
03 int g_token_line = 1;    // 当前 Token 起始行号
04 int g_token_column = 1;  // 当前 Token 起始列号
05 int g_comment_depth = 0; // 嵌套注释深度
06 bool g_in_comment = false; // 是否在注释内
07 bool g_in_directive = false; // 是否在编译器指令内
08 std::string g_last_rule;  // 最近一次命中的规则标签
09 std::string g_last_lexeme; // 最近一次识别的词素
10 std::string g_last_type;  // 最近一次识别的 Token 类型
11 std::vector<CompileError> g_lex_errors; // 词法错误列表
12 size_t g_strdup_count = 0; // strdup 次数统计
13 int g_token_count = 0;    // Token 计数
```

关键字表 g_keyword_tokens 为 std::unordered_map，将小写关键字字符串映射到 Bison Token 编号，共 33 个关键字条目，其中包含扩展关键字 type 与 record。

```
01 const std::unordered_map<std::string, int> g_keyword_tokens = {
02     {"program", PROGRAM}, {"const", CONST}, {"var", VAR}, {"integer", INTEGER},
03     {"real", REAL}, {"boolean", BOOLEAN}, {"char", CHAR}, {"array", ARRAY},
04     {"of", OF}, {"function", FUNCTION}, {"procedure", PROCEDURE},
05     {"begin", KW_BEGIN}, {"end", KW_END}, {"if", IF}, {"then", THEN},
06     {"else", ELSE}, {"while", WHILE}, {"do", DO}, {"for", FOR},
07     {"to", TO}, {"downto", DOWNTO}, {"break", BREAK}, {"read", READ}, {"write",
WRITE},
08     {"not", NOT}, {"and", AND}, {"or", OR}, {"div", DIV}, {"mod", MOD},
09     {"true", TRUE}, {"false", FALSE},
10     {"type", TYPE}, {"record", RECORD}
11 };
```



词法分析状态机与观测路径: INITIAL、COMMENT、DIRECTIVE 三类扫描状态协同工作, 并通过 YY_USER_ACTION 统一维护 Token 起始位置。

4.1.3 关键算法

标识符与关键字识别:

词法分析器遇到以字母开头的标识符模式时, 首先将词素转为小写, 然后查关键字表。若命中则返回对应关键字 Token; 否则返回 IDENTIFIER Token, 词素值为小写形式的标识符。这种“转小写后查表”的设计一次性解决了 Pascal-S 不区分大小写的问题。

标识符与关键字识别	
1	输入: 词素 <code>text</code> , 长度 <code>len</code>
2	<code>text_lower <- toLower(text, len)</code>
3	<code>if keyword_table.contains(text_lower) then</code>
4	<code>return keyword_table[text_lower]</code>
5	<code>else</code>
6	<code>return IDENTIFIER(text_lower)</code>
7	<code>end if</code>

算法伪代码: 标识符与关键字识别

位置跟踪:

通过 Flex 的 YY_USER_ACTION 宏在每个 Token 识别前保存起始位置 (行、列), 识别结束后通过 `advancePosition()` 更新行、列信息。该宏确保每个 Token 都能获得精确的 `yylloc` 位置信息。

```

01 #define YY_USER_ACTION \
02     do { \
03         g_token_line = g_line; \
04         g_token_column = g_column; \
05         yylloc.first_line = g_token_line; \
06         yylloc.first_column = g_token_column; \
07         advancePosition(yytext, yyleng); \
08         yylloc.last_line = g_line; \
09         yylloc.last_column = g_column; \
10     } while (0);

```

注释处理:

采用 Flex 的 Start Condition 机制处理注释。遇到 { 时进入 COMMENT 状态，同时 g_comment_depth 自增。在 COMMENT 状态下再次遇到 { 则嵌套深度自增，遇到 } 则自减——减至零时退出注释模式返回 INITIAL 状态。换行符在 COMMENT 状态下仍被追踪以保持行号准确。

注释中的换行符通过 COMMENT 状态的 \n 规则处理，而行号/列号的更新已在 YY_USER_ACTION 中完成。

注释嵌套处理

```

1  输入: 当前字符 ch
2  if ch = '{' then
3      g_comment_depth <- g_comment_depth + 1
4      if g_comment_depth = 1 then
5          begin COMMENT state
6      end if
7  else if ch = '}' then
8      if g_comment_depth > 0 then
9          g_comment_depth <- g_comment_depth - 1
10         if g_comment_depth = 0 then
11             begin INITIAL state
12         end if
13     end if
14 end if

```

算法伪代码: 注释嵌套处理

数值识别:

词法分析器支持多种数值格式:

- 十进制整数: 纯数字序列
- 带小数点的实数: digits.digits[exponent]
- 以 . 开头的实数: .digits[exponent]

- 科学计数法: `digits e[+-]digits`
- 十六进制数: `0x[0-9A-Fa-f]+` 和 `$(0-9A-Fa-f)+`
- 可通过 `isNumericOverflow()` 检测溢出（使用 `strtod` 函数）
- 对格式错误的数值（如多小数点）报告词法错误

字符串与字符识别:

- 字符常量: 被单引号包裹的单个字符（含转义）识别为 `CHARACTER`
- 字符串常量: 被单或双引号包裹的多个字符识别为 `STRING`
- 对未闭合的引号报告相应错误

编译器指令:

遇到 `{ $` 时进入 `DIRECTIVE` 状态, 收集指令体直到 `}`。通过 `isValidDirective()` 验证指令是否合法 (`IFDEF`、`UNDEF`、`DEFINE`、`ENDIF`、`ELSE`)。不合法指令产生词法错误但不影响编译流程。

4.1.4 关键规则片段

以下规则片段直接体现了“先大小写归一化, 再区分关键字与标识符”的实现策略:

```
01 {ID_START}{ID_CONT}* {
02   if (static_cast<size_t>(yyleng) > MAX_IDENTIFIER_LENGTH) {
03       reportLexError(g_token_line, g_token_column,
04           "identifier too long", yytext);
05       continue;
06   }
07   std::string lowered = toLower(yytext, yylen);
08   auto it = g_keyword_tokens.find(lowered);
09   if (it != g_keyword_tokens.end()) {
10       setTokenMeta("Keyword", "keyword_identifier");
11       yylval.text = duplicateText(yytext);
12       return it->second;
13   }
14   setTokenMeta("Identifier", "identifier");
15   yylval.text = duplicateText(lowered.c_str());
16   return IDENTIFIER;
17 }
```

4.1.5 错误处理

词法分析器可检测并报告以下错误类型:

- 非法字符（任何不匹配上述规则的字符）
- 未闭合的注释（文件结束时仍在 `COMMENT` 状态）

- 未闭合的编译器指令（文件结束时仍在 `DIRECTIVE` 状态）
- 未闭合的字符或字符串字面量
- 数值溢出
- 标识符过长（超过 255 字符）
- 字符串过长（超过 1024 字符）
- 格式错误的数值

所有词法错误记录在 `g_lex_errors` 向量中，通过 `lexerErrorCount()` 和 `lexerErrorAt(int)` 供外部访问。遇到错误时不终止分析，继续处理后续词素。

4.1.6 BOM 处理

在 `INITIAL` 状态下，如果是文件第一行第一列遇到 UTF-8 BOM (`\xEF\xBB\xBF`)，则静默忽略；否则报告为非法字符。

4.1.7 文件结束处理

当 Flex 遇到 `<<EOF>>` 时，检查是否有未闭合的注释或指令。如有则记录错误，清空状态，并返回 0（表示 Token 流结束）。

4.2 语法分析模块

4.2.1 输入输出

- 输入：由 `yylex()` 提供的 Token 流
- 输出： `ProgramNode*`（AST 根节点）

4.2.2 数据结构

语法分析器使用以下全局变量：

```
01 static ASTBuilder g_astBuilder;      // AST 节点构建器
02 static ProgramNode* g_parseRoot;    // 解析结果根节点
03 static int g_rule_line;             // 当前归约行号
04 static int g_rule_column;           // 当前归约列号
05 static int g_parse_error_count;     // 语法错误计数
```

Bison 使用 `%union { void* node; char* text; }` 联合体表示文法符号的语义值。AST 节点以 `void*` 形式在文法规则间传递，通过 `asNode()` / `asList()` 辅助函数转换。

所有 Bison Token 分为四类：

- **关键字 Token**： `PROGRAM`、`CONST`、`VAR`、`TYPE`、`RECORD`、`INTEGER`、`REAL`、`BOOLEAN`、`CHAR` 等共 33 个。

- **字面量 Token:** NUMBER、CHARACTER、STRING、IDENTIFIER, 携带 text 属性。
- **运算符 Token:** ASSIGN、LE、GE、NE、DOTDOT (多字符运算符), 以及单字符运算符 (+、- 等, 返回 ASCII 值)。
- **分隔符 Token:** (、)、[、]、;、,、.、: 等。

4.2.3 关键算法

AST 构建:

在每个文法产生式的归约动作中, 调用 `g_astBuilder` 的工厂方法创建 AST 节点。例如:

```
01 // 赋值语句: variable ASSIGN expression
02 $$ = static_cast
03      <void*>(g_astBuilder.makeAssignStmt(asNode($1), asNode($3), currentPos()));
```

`currentPos()` 返回当前归约规则的起始位置 (`SourcePos`), 确保每个 AST 节点都有正确的位置信息。

多维数组处理:

`buildArrayTypeFromRanges()` 函数将 `period` 列表 (逗号分隔的多个 `range`) 转化为嵌套的 `ArrayTypeNode`。遍历时从最后一个维度向前构建, 每层包裹前一层元素类型。

例如 `array[1..3, 2..4] of integer` 构建为:

```
01 ArrayType(lower=2, upper=4, elem=ArrayType(lower=1, upper=3,
elem=IdentifierNode("integer")))
```

多维数组类型构建 `buildArrayTypeFromRanges`

```
1  输入: rangesNode (period 列表), elemType (元素类型), pos (位置)
2  if rangesNode = nullptr or rangesNode is not ListNode then
3      return elemType
4  end if
5  current <- elemType
6  for i <- rangesNode.children.size() - 1 downto 0 do
7      seg <- rangesNode.children[i] as ArrayTypeNode
8      current <- makeArrayType(seg.lower, seg.upper, current, pos)
9  end for
10 return current
```

算法伪代码: 多维数组类型构建 `buildArrayTypeFromRanges`

混合变量访问处理:

buildFromArrayAccessFromIndices() 函数处理 variable → id id_varpart 中的访问链。id_varpart 可包含：

- [expression_list]: 数组下标访问（一个或多个索引表达式）。
- . IDENTIFIER: 字段访问（点号 + 字段名）。

字段名在 id_varpart 中通过创建 FieldAccessNode(base=nullptr, fieldName) 作为标记节点，与数组索引区分。此标记节点随后在 buildFromArrayAccessFromIndices 中被转换为正确的 FieldAccessNode。

混合变量访问解析 buildFromArrayAccessFromIndices

```

1  输入: base (基础标识符), indicesNode (访问链列表 ListNode), pos (位置)
2  if indicesNode = nullptr or indicesNode is not ListNode then
3      return base
4  end if
5  current <- base
6  for each child in indicesNode.children do
7      if child is FieldAccessNode (base = nullptr) then
8          current <- makeFieldAccess(current, child.fieldName, pos)
9      else
10         current <- makeArrayAccess(current, child, pos)
11     end if
12 end for
13 return current

```

算法伪代码：混合变量访问解析 buildFromArrayAccessFromIndices

运算符优先级：

语法分析器使用 Bison 的 %left/%right/%nonassoc 声明定义运算符优先级和结合性：

```

01 %right ASSIGN           // 赋值（右结合，优先级最高）
02 %nonassoc LOWER_THAN_ELSE
03 %nonassoc ELSE
04 %left OR                // 逻辑或
05 %left AND               // 逻辑与
06 %left '=' NE '<' '>' LE GE // 关系比较
07 %left '+' '-'           // 加减
08 %left '*' '/' DIV MOD   // 乘除
09 %right NOT              // 逻辑非
10 %right UMINUS           // 一元负号
11 %right UPLUS            // 一元正号

```

悬挂 else 消歧：

使用 `%prec LOWER_THAN_ELSE` 解决经典的“悬挂 `else`”歧义问题。产生式 `IF expression THEN statement %prec LOWER_THAN_ELSE` 的优先级低于 `ELSE`，从而在遇到 `else` 时选择移入（将 `else` 与最近的 `if` 匹配），而非归约。

Bison 位置传播:

通过自定义 `YYLLOC_DEFAULT` 宏实现归约时位置的正确传播:

```
01 #define YYLLOC_DEFAULT(Current, Rhs, N)
02   (Current).first_line = YYRHSLOC(Rhs, 1).first_line;
03   (Current).first_column = YYRHSLOC(Rhs, 1).first_column;
04   (Current).last_line = YYRHSLOC(Rhs, N).last_line;
05   (Current).last_column = YYRHSLOC(Rhs, N).last_column;
```

该宏使得归约产生式的起始位置为第一个符号的起始位置，结束位置为最后一个符号的结束位置，并同步更新 `g_rule_line/g_rule_column` 供 `currentPos()` 使用。

4.2.4 错误处理

语法分析器实现恐慌模式（panic mode）错误恢复，在 `statement_list` 产生式中专门增加错误恢复规则:

```
01 | statement_list ';' error
02 {
03     yyerrok;    // 恢复为正常错误状态
04     yyclearin;  // 清除预读 Token
05     $$ = $1;    // 继续使用当前语句列表
06 }
```

当在语句列表（分号后）遇到语法错误时，Bison 丢弃后续 Token 直到遇到分号，然后通过 `yyerrok` 恢复正常状态，继续分析后续语句。这种设计确保一个语句的语法错误不会导致整个程序的解析失败。

错误信息通过 `yyerror()` 函数输出，格式为:

```
01 Parse error at 行:列 near 'lexeme': 错误描述
```

4.2.5 语法规则覆盖

语法分析器实现的文法产生式覆盖:

1. 程序结构: `program → PROGRAM IDENTIFIER opt_program_input ';' program_body '.'`
2. 常量声明: `const_declarations → CONST const_declaration_list ';'`

3. **类型声明 (扩展):** `type_declarations` $\rightarrow$ `TYPE type_declaration_list ';' ;`, 支持 `record` 类型定义
4. **变量声明:** `var_declarations` $\rightarrow$ `VAR var_declaration_list ';' ;`, 类型支持基本类型、数组和用户自定义类型
5. **子程序声明:** `subprogram` $\rightarrow$ `PROCEDURE/FUNCTION IDENTIFIER formal_parameter ...`, 参数支持 `var` 和值参数
6. **语句:** 赋值、过程调用、复合语句、条件语句、`for/while` 循环、`break`、`read/write`
7. **表达式:** 支持算术、关系、逻辑运算符, 函数调用, 数组访问, 字段访问

4.3 AST 模块

4.3.1 概述

AST (抽象语法树) 模块是编译器所有后续阶段共享的核心数据结构。AST 节点定义在 `include/ast.h` 中, 构建器实现在 `src/ast.cpp` 中。整个 AST 体系基于面向对象的多态设计, 基类 `ASTNode` 定义了所有节点共有的属性和接口。

4.3.2 数据类型

`DataType` 枚举定义在 `include/common.h` 中:

```
01 enum class DataType {
02     Integer,    // 整型
03     Real,       // 实型
04     Boolean,    // 布尔型
05     Char,       // 字符型
06     Procedure,  // 过程
07     Function,   // 函数
08     Record,     // record 类型 (扩展)
09     Unknown     // 未知 (初始值)
10 };
```

4.3.3 节点位置

`SourcePos` 结构记录节点在源码中的位置:

```
01 struct SourcePos {
02     int line = 0; // 行号
03     int col = 0;  // 列号
04 };
```

该信息在语法分析归约阶段由 `currentPos()` 提供，确保错误报告能精确定位到源码位置。

4.3.4 核心节点详解

ProgramNode: 程序根节点，包含 `name`（程序名）和 `children`（子节点为 `BlockNode`，可选的程序头标识符列表）。该节点在整个 AST 中只有一个实例。

BlockNode: 作用域块节点。程序体（`program_body`）的 `BlockNode` 包含 5 个子节点：`[consts, types, vars, subprograms, compound]`；子程序体（`subprogram_body`）的 `BlockNode` 包含 3 个子节点：`[consts, vars, compound]`。

VarDeclNode: 变量声明节点，`children[0]` 为标识符列表（`ListNode`），`children[1]` 为类型节点。一个变量声明可同时声明多个同类型变量（如 `a, b, c : integer`）。

ConstDeclNode: 常量声明节点，`children[0]` 为标识符，`children[1]` 为常量值（可以是 `LiteralNode` 或 `UnaryExprNode`，如负号表达式）。

ProcDeclNode / FuncDeclNode: 过程/函数声明节点，包含名称、参数列表（`children[0]`）和子程序体（`children[1]`）。`FuncDeclNode` 额外包含 `retType` 字段记录返回值类型。

ProcCallNode: 过程/函数调用节点，包含名称（`name`）、实参列表（`children`）、`isVarParam` 标识列表（标记哪些实参需按引用传递）、`builtinKind`（标记内置过程 `read/write`）。

ForStmtNode: `for` 循环语句节点，包含 `isDownto` 标志（区分 `to` 和 `downto`），`children` 为 `[id, init, end, body]`。

BinaryExprNode / UnaryExprNode: 表达式节点，通过 `op` 字符串存储运算符。支持的二元运算符：`+`、`-`、`*`、`/`、`div`、`mod`、`and`、`or`、`=`、`<>`、`<`、`>`、`<=`、`>=`。支持的一元运算符：`-`、`not`。

IdentifierNode: 标识符引用节点，包含 `identifier` 字符串（已转小写）、`isLValue` 标记（是否可作为左值）、`isFunctionResultTarget` 标记（是否是函数结果赋值的左侧）。

LiteralNode: 字面量节点，包含 value 字符串（字面量文本，如 "3.14"、"'a'"）、isStringLikeLiteral 标记（类字符串常量，用于 write 语句的 %s 格式符）。

ArrayAccessNode: 数组访问节点，children[0] 为基础表达式（数组变量），children[1] 为下标表达式。多维数组通过嵌套 ArrayAccessNode 表示。代码生成时利用 symbolEntry 中的 arrayBounds 计算下标偏移。

ArrayTypeNode: 数组类型节点，children[0] 为下界字面量，children[1] 为上界字面量，children[2] 为元素类型节点。多维数组通过嵌套 ArrayTypeNode 表示（每层包裹内层类型）。

CompoundStmtNode: 复合语句节点（对应于 begin ... end），children 为语句列表。

ListNode: 通用列表容器节点。不同于其他节点，ListNode 的 children 动态添加。ListKind 字段指示列表语义：

ListKind	用途
Unknown	未指定
Identifiers	标识符列表（idlist: 逗号分隔的标识符）
Statements	语句列表（statement_list）
Expressions	表达式列表（expression_list，含 variable_list）
Parameters	参数列表（parameter_list）
Declarations	声明列表（常量声明、变量声明、子程序声明）
ArrayRanges	多维数组的 period 列表（逗号分隔的 range）
FieldAccess	字段访问链标记（用于区分数组下标和字段访问）

AssignStmtNode: 赋值语句节点，children[0] 为左值（变量/字段访问），children[1] 为右值表达式。支持普通赋值和函数结果赋值两种语义，后者通过左值的 isFunctionResultTarget 标记区分。

IfStmtNode: 条件语句节点，children[0] 为条件表达式，children[1] 为 then 分支，children[2] 为可选的 else 分支（可为 nullptr）。

WhileStmtNode: while 循环语句节点，children[0] 为循环条件，children[1] 为循环体语句。

BreakStmtNode: break 语句节点，无子节点。在语义分析阶段通过 loopDepth_ 计数器验证其必须出现在循环体内。

ParamDeclNode: 参数声明节点, children[0] 为标识符列表, children[1] 为类型节点, isVar 字段标记是否为 var 引用参数。

TypeDeclNode: 类型声明节点 (type name = type_definition;), name 存储类型名, children[0] 指向类型定义节点 (如 RecordTypeNode 或基本类型标识符)。

RecordTypeNode: record 类型定义节点, children[0] 指向字段列表 (ListNode), 每个字段为 FieldDeclNode。

FieldDeclNode: 字段声明节点, children[0] 为标识符列表 (单个字段名), children[1] 为类型节点 (基本类型)。

FieldAccessNode: 字段访问节点 (base.fieldName), children[0] 为基础表达式, fieldName 存储字段名。语义分析阶段设置为可作赋值左值, 代码生成阶段翻译为 C 的点运算符。

4.3.5 Arena 内存管理

ASTBuilder 类内部维护 `std::vector<std::unique_ptr<ASTNode>> arena_`, 所有通过 `create<T>(...)` 模板方法创建的节点均存储在 Arena 中。这种设计优势:

- 节点生命周期与 ASTBuilder 或 AST 根节点一致, 无需手动释放。
- 节点指针在 Arena 生命周期内保持稳定。
- 支持 `resetParseResult()` 销毁整个 AST (创建新的 ASTBuilder 实例, 旧 Arena 自动析构)。

4.3.6 Visitor 模式

ASTVisitor 是纯虚基类, 为 25 种节点类型各定义了一个 `visit()` 方法。这使编译器能够在不修改 AST 节点类的情况下添加新的遍历逻辑。

SemanticAnnotator 和 CodeGenerator 分别实现该接口, 前者负责语义注解, 后者负责代码生成。两者互不干扰, 实现了关注点分离。

4.4 符号表模块

4.4.1 输入输出

- 输入: 通过 `insert()` 方法接收符号条目, 通过 `lookup()` 方法查询。
- 输出: 符号查询结果 (`const SymbolEntry*`), 插入成功/失败的布尔值。

4.4.2 数据结构

符号表采用 `std::vector<std::unordered_map<std::string, const SymbolEntry*>>` 作为底层容器。每个 `unordered_map` 代表一个作用域，键为归一化（小写）的符号名，值为指向 `SymbolEntry` 的指针。

```
01 class SymbolTable {
02 public:
03     SymbolTable();
04
05     // 进入一个新的嵌套作用域
06     void enterScope();
07
08     // 退出当前作用域（移除该层所有符号）
09     void exitScope();
10
11     // 在当前作用域插入一个符号条目，若当前作用域已有同名符号则返回 false
12     bool insert(SymbolEntry entry);
13
14     // 返回当前作用域的嵌套深度（最外层为 0）
15     int currentScopeLevel() const;
16
17     // 从当前作用域向外逐层查找符号，返回找到的符号指针，未找到返回 nullptr
18     const SymbolEntry* lookup(const std::string& name) const;
19
20     // 仅在当前作用域中查找符号，不向外层查找
21     const SymbolEntry* lookupCurrentScope(const std::string& name) const;
22
23 private:
24     // 标准化符号名（例如统一小写化、去除空白等）
25     static std::string normalizeName(const std::string& name);
26
27     // 作用域栈，每个元素是当前作用域的符号名到符号条目的映射
28     std::vector<std::unordered_map<std::string, const SymbolEntry*>> scopes_;
29
30     // 符号条目的 Arena，用于管理符号条目的生命周期
31     std::vector<std::unique_ptr<SymbolEntry>> entryArena_;
32 };
```

符号名称归一化通过 `normalizeName()` 静态方法实现，将所有字符转换为小写：

```
01 static std::string normalizeName(const std::string& name) {
02     std::string normalized;
03     for (char ch : name) {
04         normalized += std::tolower(static_cast<unsigned char>(ch));
05     }
06 }
```

```
06     return normalized;
07 }
```

符号条目 `SymbolEntry` 的字段类型 `SymbolKind` 枚举:

```
01 enum class SymbolKind {
02     Variable,    // 变量
03     Constant,    // 常量
04     Procedure,   // 过程
05     Function,    // 函数
06     Parameter,   // 参数
07     TypeAlias    // 用户自定义类型别名
08 };
```

参数/字段信息 `ParamInfo` 结构:

```
01 struct ParamInfo {
02     std::string name;
03     DataType type = DataType::Unknown;
04     bool isVarParam = false; // 默认不是var引用参数
05 };
```

数组边界 `ArrayBound` 结构:

```
01 struct ArrayBound {
02     int lower = 0;
03     int upper = -1;
04 };
```

4.4.3 关键算法

插入操作 (insert)

将符号插入当前作用域。首先调用 `normalizeName()` 归一化名称，然后检查当前作用域是否已存在同名符号。若存在则返回 `false`，表示重复定义。否则将条目指针存入当前作用域的哈希表中。

条目本身存储于 `entryArena_` 中，确保指针稳定性:

```
01 bool SymbolTable::insert(SymbolEntry entry) {
02     std::string key = normalizeName(entry.name);
03     auto& current = scopes_.back();
04     if (current.find(key) != current.end()) {
05         return false; // 当前作用域中重复定义
06     }
07     auto unique = std::make_unique<SymbolEntry>(std::move(entry));
08     const SymbolEntry* raw = unique.get();
09     entryArena_.push_back(std::move(unique));
10     current[key] = raw;
```

```

11     return true;
12 }

```

查找操作 (lookup)

从最内层作用域向最外层方向查找。lookup() 从 scopes_ 的末尾向前遍历 (对应从局部到全局)，在每个作用域的哈希表中查找归一化后的名称。找到则返回 SymbolEntry*，找不到返回 nullptr。

```

01 const SymbolEntry* SymbolTable::lookup(const std::string& name) const {
02     std::string key = normalizeName(name);
03     for (auto it = scopes_.rbegin(); it != scopes_.rend(); ++it) {
04         auto found = it->find(key);
05         if (found != it->end()) {
06             return found->second;
07         }
08     }
09     return nullptr;
10 }

```

lookupCurrentScope() 仅查当前最内层作用域，用于重复定义检测。

符号表查找 lookup

```

1  输入: name (原始符号名)
2  key <- normalizeName(name)  // 统一转小写
3  for scope in reverse(scopes) from innermost to outermost do
4      if scope.contains(key) then
5          return scope[key]
6      end if
7  end for
8  return nullptr  // 未找到

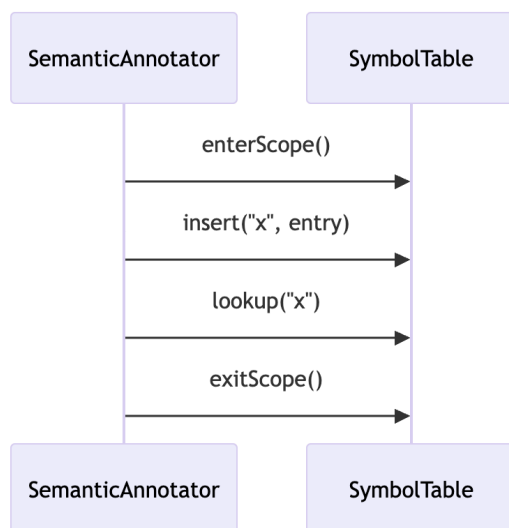
```

算法伪代码: 符号表查找 lookup

作用域管理

enterScope(): 向 scopes_ 尾部追加一个空 unordered_map，表示进入新作用域。由于过程/函数不可嵌套定义，最大作用域深度为 2 (全局 + 局部)。

exitScope(): 弹出 scopes_ 尾部元素。注意条目对象保留在 entryArena_ 中，因此退出作用域后已注册的符号指针仍然有效。



进入与退出作用域示例

4.4.4 内置符号注册

在语义分析开始前，编译器的 `semantic_register::preregisterBuiltins(symbolTable)` 函数注册以下内置符号：

- `read`：内置过程，参数个数可变、类型可变。
- `write`：内置过程，参数个数可变、类型可变。
- `true` 和 `false`：布尔常量，由 `SemanticAnnotator` 的构造函数注册。

4.5 语义分析模块

4.5.1 输入输出

- 输入：语法分析阶段输出的 `ASTNode*`（AST 根节点）。
- 输出：注解后的 AST——每个节点填充 `dataType` 和 `symbolEntry`，同时通过 `ErrorHandler` 报告语义错误。

4.5.2 数据结构

语义分析器维护以下上下文状态：

```

01 SymbolTable& symbolTable_;           // 符号表引用
02 ErrorHandler& errorHandler_;        // 错误处理器引用
03 std::vector<std::string> functionContextStack_; // 当前函数上下文栈
04 int loopDepth_ = 0;                  // 当前循环嵌套深度
05 int valueContextDepth_ = 0;          // 值上下文深度
  
```


- `functionContextStack_`: 函数名栈，用于检测函数结果赋值是否出现在正确的函数体中。
- `loopDepth_`: 记录循环嵌套深度，`break` 语句验证（必须 >0 ）。
- `valueContextDepth_`: 标记当前表达式的求值上下文。当表达式作为右值时（如赋值语句右值、参数等），通过 `annotateValueNode()` 推进；作为左值时则不推进。

4.5.3 关键算法

注解调度（`annotateNode`）

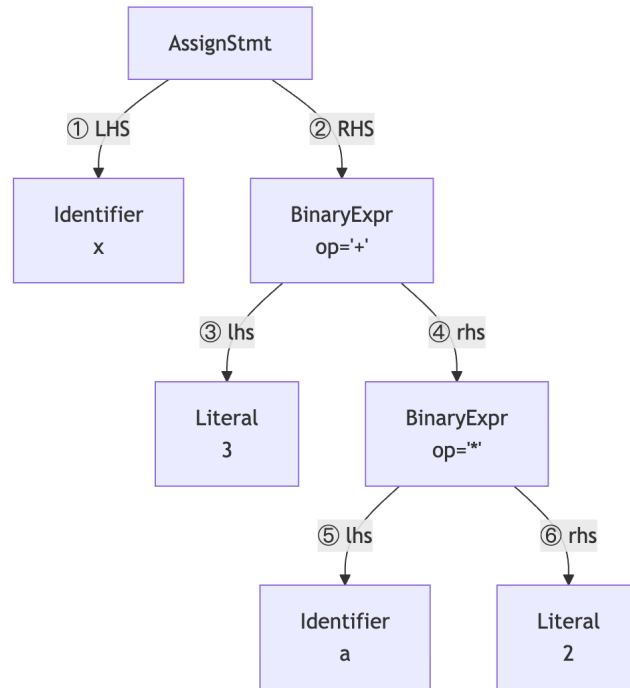
`annotateNode()` 是语义分析的核心分发器，根据节点的 `nodeType` 派发到对应的 `annotate*` 方法：

```

01 void SemanticAnnotator::annotateNode(ASTNode* node) {
02     switch (node->nodeType) {
03         case NodeType::Program:      annotateProgram(...); break;
04         case NodeType::Block:        annotateBlock(...);   break;
05         case NodeType::VarDecl:      annotateVarDecl(...);  break;
06         case NodeType::ConstDecl:    annotateConstDecl(...); break;
07         case NodeType::ProcDecl:     annotateProcDecl(...); break;
08         case NodeType::FuncDecl:     annotateFuncDecl(...); break;
09         // ... 所有 25 种节点类型
10     }
11 }

```

以语句 `x := 3 + a * 2`（假设 `x`、`a` 已声明为 `integer`）为例，其 AST 结构及语义分析的深度优先遍历顺序如下图所示（边上的 ①-⑥ 为遍历次序）：



$x := 3 + a * 2$ 的 AST 结构示意图

各步骤注解过程说明：

- ① **annotateNode(lhs) → annotateIdentifier(x)**: 查找符号表 `lookup('x')` → 绑定 `symbolEntry=Var(x,Integer)`, 设置 `isLValue=true`。
- ② **annotateValueNode(rhs) → annotateBinaryExpr(+)**: 进入二元表达式, 对左右操作数递归调用 `annotateValueNode`。
- ③ **annotateLiteral(3)**: 纯数字字面量 → `dataType=Integer`。返回到 ②。
- ④ **annotateValueNode(rhs_of_+) → annotateBinaryExpr(*)**: 嵌套乘法表达式, 继续递归左右操作数。
- ⑤ **annotateIdentifier(a)**: 查找符号表 → `dataType=Integer`。返回到 ④。
- ⑥ **annotateLiteral(2)**: 纯数字 → `dataType=Integer`。返回到 ④。
- 回溯到 ④: 检查 `isNumeric(Integer) ∧ isNumeric(Integer)` → 通过, `dataType=Integer`。返回到 ②。
- 回溯到 ②: 检查 `isNumeric(Integer) ∧ isNumeric(Integer)` → 通过, `dataType=Integer`。返回到 `AssignStmt`。
- 回溯到 `AssignStmt`: 调用 `isAssignable(lhs=Integer, rhs=Integer)` → `lhs==rhs`, 通过, 注解完成。

遍历整体遵循**深度优先、自底向上**的求值顺序：叶子节点先注解类型，父节点在子节点全部返回后再执行类型检查并推导自身类型。

类型推导 (inferType / inferTypeFromTypeNode)

inferType(node) 根据字面量内容或已查询的符号条目前向推导节点类型：

- true/false → Boolean
- 单字符常数（如 'a'）→ Char
- 含小数点或科学计数法的数字 → Real
- 纯整数 → Integer
- 已绑定 symbolEntry 的标识符 → symbolEntry->type

inferTypeFromTypeNode(node) 专门用于类型声明节点（出现在 : type 中），首先检查基本的 IdentifierNode（如 "integer"），若非基本类型则查询符号表查找用户自定义类型（如 record 类型）。

类型推导 inferType	
1	输入: node (AST 节点)
2	if node is LiteralNode then
3	if node.value = "true" or node.value = "false" then return Boolean
4	if node.value is character literal then return Char
5	if node.value contains '.' or 'e' then return Real
6	else return Integer
7	end if
8	if node is IdentifierNode then
9	entry <- symbolTable.lookup(node.identifier)
10	if entry != nullptr then
11	node.symbolEntry <- entry
12	return entry.type
13	else report "undeclared identifier"
14	end if
15	return Unknown

算法伪代码：类型推导 inferType

声明处理

- annotateConstDecl: 注解常量值表达式，调用 inferType() 推导类型，创建 SymbolEntry::makeConstant() 插入符号表，记录常量字面量文本。

- `annotateVarDecl`: 从类型节点推导变量类型，收集数组边界信息。处理标识符列表中的每个标识符，依次注册。支持用户自定义类型（record）——通过查找符号表中的 `TypeAlias` 获取 `typeName` 和 `fields`。
- `annotateParamDecl`: 类似变量声明处理，同时处理 `var` 参数标记。
- `annotateProcDecl`: 将自身名称注册为过程符号，收集参数信息（`SymbolEntry.params`），`enterScope()` 后注解子程序体，最后 `exitScope()`。
- `annotateFuncDecl`: 类似过程声明，额外记录返回类型。注解子程序体前将函数名推入 `functionContextStack_`。

表达式类型检查

`annotateBinaryExpr` 根据运算符类型对操作数实施不同的类型约束：

运算符	操作数约束	结果类型
<code>+ - *</code>	操作数为 <code>numeric</code> (<code>integer/real</code>)	同操作数类型，除法返回 <code>Real</code>
<code>div mod</code>	操作数均为 <code>integer</code>	<code>Integer</code>
<code>and or</code>	操作数均为 <code>boolean</code>	<code>Boolean</code>
<code>= < < > <= >=</code>	操作数类型一致	<code>Boolean</code>

`annotateUnaryExpr`: `not` 要求操作数为 `Boolean` 或 `Integer`，一元 `-` 要求操作数为 `numeric`。

以下为表达式类型检查的核心代码逻辑：

```

01 void SemanticAnnotator::annotateBinaryExpr(BinaryExprNode* node) {
02     DataType lt = inferType(lhs), rt = inferType(rhs);
03     if (op == "+" || op == "-" || op == "*") {
04         if (!isNumericType(lt) || !isNumericType(rt))
05             report error "Arithmetic operator requires numeric operands";
06         node->dataType = (lt == Real || rt == Real) ? Real : Integer;
07     } else if (op == "div" || op == "mod") {
08         if (!isIntegerType(lt) || !isIntegerType(rt))
09             report error "div/mod operator requires integer operands";
10         node->dataType = Integer;
11     } else if (op == "and" || op == "or") {
12         if (!isBooleanType(lt) || !isBooleanType(rt))
13             report error "Logical operator requires boolean operands";
14         node->dataType = Boolean;
15     }

```

```
16 // 关系运算符：比较类型一致性，结果返回 Boolean
17 }
```

标识符注解

`annotateIdentifier` 查询符号表，若符号不存在则报告“未声明标识符”错误。查询成功后，将节点的 `symbolEntry`、`dataType` 设置为查询结果，并根据符号种类设置 `isLValue`（变量和参数为 `true`，常量为 `false`）。

特殊处理：若当前处于值上下文中且符号是函数，将 `isFunctionResultTarget` 设为 `true`，代码生成时将生成函数调用表达式。

```
01 void SemanticAnnotator::annotateIdentifier(IdentifierNode* node) {
02     const SymbolEntry* entry = symbolTable_.lookup(node->identifier);
03     if (entry == nullptr) {
04         report error "Undefined identifier: " + node->identifier;
05         node->dataType = Unknown;
06         return;
07     }
08     node->symbolEntry = entry;
09     node->dataType = entry->type;
10     node->isLValue = (entry->kind == Variable || entry->kind == Parameter);
11 }
```

数组访问检查

`annotateArrayAccess` 验证下标表达式为 `integer` 类型，基础表达式为数组类型。利用 `tryEvalIntConst()` 进行编译期常量求值，检查下标是否在 `[lower, upper]` 范围内。为支持后续 `record` 字段访问，将 `symbolEntry` 传递给 `ArrayAccessNode`。

过程/函数调用检查

`checkCallArguments()` 验证过程/函数调用的参数数量和类型：

- `read`：所有参数必须为左值（需要存储读入的值）。
- `write`：任意表达式均可。
- 用户自定义过程/函数：参数个数匹配，类型兼容（值参数需 `isAssignable`，`var` 参数需 `isLValue` 且类型完全匹配）。

```
01 void SemanticAnnotator::checkCallArguments(ProcCallNode* node, const
SymbolEntry* callee) {
02     if (callee->name == "read") {
03         for each arg: if (!isLValue(arg))
04             report error "read expects assignable variable";
```

```

05     return;
06 }
07 if (expected != actual) // 参数个数检查
08     report error "Argument count mismatch: expected X, got Y";
09 for each arg:
10     if (!isAssignable(param.type, actualType)) // 类型兼容
11         report error "Argument type mismatch";
12     if (param.isVarParam && !isLValue(arg)) // var 参数左值检查
13         report error "var parameter requires assignable argument";
14 }

```

赋值语句检查

annotateAssignStmt 处理两种赋值：

1. 常规赋值：左侧必须是可赋值的（左值或字段访问），类型兼容检查（允许 integer→real 隐式转换）。
2. 函数结果赋值：左侧标识符的 isFunctionResultTarget 为 true，且函数名与当前函数上下文栈顶一致。

类型兼容性由 isAssignable() 判定——允许 integer → real 的拓宽转换，其余类型要求完全匹配：

```

01 bool SemanticAnnotator::isAssignable(DataType lhs, DataType rhs) const {
02     if (lhs == Unknown || rhs == Unknown) return false;
03     if (lhs == Real && rhs == Integer) return true; // 拓宽转换
04     return lhs == rhs;
05 }

```

循环上下文与 break 检查

annotateWhileStmt、annotateForStmt 进入前 loopDepth_++，退出后 loopDepth_--。annotateBreakStmt 验证 loopDepth_ > 0，若不在循环内则报告错误。

```

01 void SemanticAnnotator::annotateBreakStmt(BreakStmtNode* node) {
02     if (loopDepth_ <= 0)
03         report error "break can only be used inside while/for loop";
04 }
05
06 void SemanticAnnotator::annotateWhileStmt(WhileStmtNode* node) {
07     if (!isBooleanType(inferType(cond)))
08         report error "while condition must be boolean";
09     loopDepth_++;
10     annotateNode(body);
11     loopDepth_--;
12 }

```

常量表达式求值

`tryEvalIntConst()` 尝试在编译期计算整型常量表达式的值，支持：

- 字面量（整数、字符）
- 一元运算（+、-）
- 二元运算（+、-、*、div、mod）

该函数主要用于数组边界检查——当数组下标或边界为常量时，在编译期即可检测越界错误，提供精确的错误消息。

编译期常量求值 `tryEvalIntConst`

```
1  输入: node (AST 节点), result (输出参数)
2  if node is LiteralNode then
3      if node is integer literal then
4          result <- strToInt(node.value), return true
5      else if node is character literal then
6          result <- charToInt(node.value), return true
7      end if
8  end if
9  if node is UnaryExprNode then
10     if node.op = '-' and tryEvalIntConst(node.child, value) then
11         result <- -value, return true
12     if node.op = '+' and tryEvalIntConst(node.child, value) then
13         result <- value, return true
14 end if
15 if node is BinaryExprNode then
16     if tryEvalIntConst(node.lhs, lv) and tryEvalIntConst(node.rhs,
17         rv) then
18         result <- evaluate(lv, node.op, rv), return true
19 end if
19 return false // 非常量表达式
```

算法伪代码：编译期常量求值 `tryEvalIntConst`

实际实现中，`tryEvalIntConst` 递归遍历字面量、一元表达式和二元表达式进行编译期求值，对 `div/mod` 额外检查除零错误：

```
01 bool SemanticAnnotator::tryEvalIntConst(ASTNode* node, int& out) const {
02     if (auto* lit = dynamic_cast<LiteralNode*>(node)) {
03         out = std::stoi(lit->value); // 仅全数字字面量
04         return true;
05     }
06     if (auto* un = dynamic_cast<UnaryExprNode*>(node)) {
07         int v; if (tryEvalIntConst(un->children[0], v)) {
08             out = (un->op == "-") ? -v : v; return true;
09         }
```

```

09     }
10 }
11 if (auto* bin = dynamic_cast<BinaryExprNode*>(node)) {
12     int lv, rv;
13     if (tryEvalIntConst(bin->children[0], lv)
14         && tryEvalIntConst(bin->children[1], rv)) {
15         if (bin->op == "+") out = lv + rv;      // 加减乘
16         else if (bin->op == "-") out = lv - rv;
17         else if (bin->op == "*") out = lv * rv;
18         else if (bin->op == "div") { if (rv == 0) return false; out = lv /
rv; }
19         else if (bin->op == "mod") { if (rv == 0) return false; out = lv %
rv; }
20         return true;
21     }
22 }
23 return false;
24 }

```

4.5.4 错误处理

语义分析器通过 ErrorHandler 报告以下语义错误：

- 未声明的标识符
- 重复定义的标识符（同一作用域内）
- 类型不匹配（赋值、表达式、参数传递）
- 数组下标非整数类型
- 下标越界（编译期可检测的）
- 对非数组类型使用下标访问
- 对非 record 类型使用字段访问
- 不存在的字段名
- break 出现在循环之外
- 函数结果赋值不匹配
- 参数个数不匹配
- read 的参数不是左值
- var 参数要求实参为左值

遇到错误时不中断分析，继续遍历 AST 以收集更多错误信息。

4.6 代码生成模块

4.6.1 输入输出与总体架构

代码生成模块的输入是语义分析后带类型注解和符号表绑定的 AST (ProgramNode*), 输出是完整的 C 语言源程序字符串。

代码生成器 CodeGenerator 采用 Visitor 模式遍历 AST。每个 visit 方法处理一种 AST 节点, 将生成的 C 代码片段写入 currentExpr_ (std::string), 供父节点组合使用。CodeGenUtils 提供一组静态工具方法用于生成声明、读写语句和最终组装。

4.6.2 四区代码缓冲

代码生成器为不同性质的声明将代码分流到四个缓冲区:

```
01 std::string globalDecls_; // 全局常量 (const)、类型定义 (typedef struct)、全局变量
02 std::string prototypes_; // 函数/过程的前置声明
03 std::string definitions_; // 函数/过程的完整定义 (含函数体)
04 std::string mainBody_; // main 函数体中的语句
```

generate() 入口方法手动解析 BlockNode 的子节点而非递归遍历, 将每个子节点类别精确路由到正确的缓冲区:

```
01 std::string CodeGenerator::generate(ProgramNode* root) {
02     reset();
03     // 程序 (program) → 包含一个 BlockNode
04     auto* block = dynamic_cast<BlockNode*>(root->children[0]);
05
06     // children[0]: 常量声明 → 全局声明区
07     for (auto* declNode : block->children[0]->children)
08         globalDecls_ += CodegenUtils::emitConstDecl(declNode);
09
10     // children[1]: 类型声明 (record 等) → 全局声明区
11     for (auto* typeNode : block->children[1]->children)
12         globalDecls_ += emitNode(typeNode);
13
14     // children[2]: 变量声明 → 全局声明区
15     for (auto* varNode : block->children[2]->children)
16         globalDecls_ += CodegenUtils::emitVarDecl(varNode);
17
18     // children[3]: 子程序 → 原型区 + 定义区
19     for (auto* subNode : block->children[3]->children) {
20         if (auto* proc = dynamic_cast<ProcDeclNode*>(subNode)) {
21             prototypes_ += CodegenUtils::emitProcPrototype(proc);
22             definitions_ += CodegenUtils::emitProcDecl(proc, *this);
23         } else if (auto* func = dynamic_cast<FuncDeclNode*>(subNode)) {
24             prototypes_ += CodegenUtils::emitFuncPrototype(func);
25             definitions_ += CodegenUtils::emitFuncDecl(func, *this);
26         }
27     }
```

```

28
29     // children[4]: 主程序体 → main 函数体
30     visit(block->children[4]);
31     mainBody_ = currentExpr_;
32
33     return CodegenUtils::wrapAsCProgram(globalDecls_, prototypes_,
34                                         definitions_, mainBody_);
35 }

```

四个缓冲区依次拼接，最终由 wrapAsCProgram() 组装为完整 C 文件：

```

01 #include <stdio.h>
02 [globalDecls_]      // const int N = 100;
03                     // typedef struct { ... } Person;
04                     // int a, b;
05 [prototypes_]       // void swap(int *x, int *y);
06                     // int gcd(int a, int b);
07 [definitions_]      // void swap(int *x, int *y) { ... }
08                     // int gcd(int a, int b) { ... }
09 int main(void) {
10     [mainBody_]
11     return 0;
12 }

```

4.6.3 类型映射

Pascal-S 基本类型到 C 类型的一一映射由 mapType() 完成：

```

01 DataType::Integer → "int"
02 DataType::Real    → "float"    // Pascal real 映射为 C float
03 DataType::Boolean → "int"      // boolean 使用 0/1 表示
04 DataType::Char    → "char"
05 default           → "int"

```

读写语句中的格式符选择同样基于类型：

```

01 DataType::Integer → "%d"
02 DataType::Real    → "%f"
03 DataType::Boolean → "%d"
04 DataType::Char    → "%c"

```

4.6.4 变量声明生成

emitVarDecl() 处理三种变量声明场景：

基本类型： 从 VarDeclNode 中提取变量名列表和类型节点，生成 int a, b, c; 形式的声明。

数组类型： 遍历嵌套的 `ArrayTypeNode` 链，逐维收集下界和上界（从 `ArrayTypeNode` 的字面量子节点中读取），计算每维大小 `upper - lower + 1`，生成 `type name[N1][N2]...`。例如，Pascal 声明 `arr: array[1..10, 2..8] of integer` 生成 C 声明 `int arr[10][7];`。

```
01 // 遍历数组类型链，收集每一维的大小
02 while (typeNode->nodeType == NodeType::ArrayType) {
03     int lower = std::stoi(asLiteral(typeNode->children[0])>value);
04     int upper = std::stoi(asLiteral(typeNode->children[1])>value);
05     dimensions.push_back(upper - lower + 1);
06     typeNode = typeNode->children[2]; // 进入下一维或元素类型
07 }
08 // 生成: type name[dims[0]][dims[1]]...
```

Record 类型： 从符号表条目获取用户定义的类型名（如 `person`），直接使用该名称 — 不参与 `mapType()` 映射。

4.6.5 常量声明生成

`emitConstDecl()` 生成 `const type name = value;`。特殊处理包括布尔常量（`true`→1, `false`→0）、负数常量（识别 `UnaryExprNode` 包裹的字面量，输出 -<value>），以及字符串类常量（使用 `const char*` 并通过 `pascalCharLiteralToCString()` 将 Pascal 单引号字符常量转换为 C 双引号字符串）。常量命名时自动避免与 C 保留字冲突（如 `true_reserved`、`false_reserved`）。

4.6.6 过程与函数翻译

4.6.6.1. 原型声明

原型生成分为过程原型和函数原型，二者的区别在于返回类型：

```
01 // 过程原型
02 void procedureName(int param1, int *param2);
03 // 函数原型
04 int functionName(int param, float *varParam);
```

参数列表的生成需处理 Pascal 的多变量声明语法：`var a, b, c: integer` 允许在一个参数组中声明多个同类型变量。遍历时对每个变量名单独生成参数条目。`var` 参数在类型前插入 `*`（指针）。

4.6.6.2. 过程定义

`emitProcDecl()` 生成完整的 `void name(params) { body }` 函数定义。函数体的生成通过调用者传入的 `CodeGenerator` 引用：`cg.visit(body)` 遍历过程体的 `BlockNode`，将局部常量、局部变量和语句依次生成为缩进后的 C 代码。

4.6.6.3. 函数定义与 `_retval` 机制

`emitFuncDecl()` 是函数翻译的核心。Pascal 语法中函数通过“对函数名赋值”来设置返回值（如 `gcd := a`），而 C 通过 `return` 语句返回。为此，引入一个名为 `_retval` 的局部变量作为中间桥梁：

```
01 int funcName(int param) {
02     int _retval;           // 1. 声明同类型的返回值临时变量
03     // ...
04     _retval = expr;        // 2. 对函数名的赋值转化为对 _retval 的赋值
05     // ...
06     return _retval;       // 3. 函数末尾返回 _retval
07 }
```

`_retval` 机制涉及三个环节的配合：

- **声明：**`emitFuncDecl()` 在函数体开头插入 `int _retval;`（类型由函数返回类型决定）。
- **赋值：**当 `AssignStmtNode` 检测到左值为一个函数标识符（`symbolEntry->kind == SymbolKind::Function`）时，生成 `_retval = <rhs>;`；而非 `funcName = <rhs>;`。
- **返回：**`emitFuncDecl()` 在函数体末尾追加 `return _retval;`。
- **read 读入：**当 `emitReadStmt()` 检测到实参标记 `isFunctionResultTarget` 时，生成 `&_retval` 作为 `scanf` 的目标地址，使 `read(f)` 能够将输入值直接写入函数返回值。

4.6.7 var 引用参数的指针翻译

Pascal 的 `var` 参数表示引用传递，在 C 中通过指针实现。这一翻译涉及三个方面：

声明侧——参数类型加 *：过程原型 `procedure swap(var x, y: integer)` 生成 C 原型 `void swap(int *x, int *y);`。

使用侧——自动解引用：在 `IdentifierNode` 的 `visit` 中，检查符号表条目的 `isVarParam` 标记。若为 `var` 参数，在标识符外包裹 `(*)`：

```

01 void CodeGenerator::visit(IdentifierNode* node) {
02     if (node->symbolEntry->kind == SymbolKind::Parameter
03         && node->symbolEntry->isVarParam) {
04         currentExpr_ = "(" + node->identifier + ")";
05         return;
06     }
07     currentExpr_ = node->identifier;
08 }

```

因此，Pascal 过程体中的 $x := x + 1$ (x 为 var 参数) 生成 C 代码 $(*x) = (*x) + 1$ ；。

调用侧——自动取址：在 ProcCallNode 的 visit 中，检查参数位置的 isVarParam[i] 标记。若对应形参为 var，则在实参前添加 &：

```

01 if (node->isVarParam[i]) {
02     currentExpr_ += "&" + argExpr;
03 } else {
04     currentExpr_ += argExpr;
05 }

```

调用 swap(a, b) 生成 swap(&a, &b);。

避免双重取址：当一个 var 参数本身作为另一个 var 参数传入时，它已经是指针，不应再加 &。代码通过检查实参 IdentifierNode 的 isVarParam 属性来识别此情况，直接传递变量名。

4.6.8 数组访问的下标偏移

Pascal 数组可以从任意整数起始（如 array[3..9] of integer），而 C 数组总是从 0 开始。visit(ArrayAccessNode*) 必须为每次数组访问生成下标偏移：

```

01 void CodeGenerator::visit(ArrayAccessNode* node) {
02     std::string base = emitNode(node->children[0]); // 数组名或上层 ArrayAccess
03     std::string index = emitNode(node->children[1]); // 下标表达式
04
05     // 从符号表获取当前维度的下界
06     int lowerBound = node->lowerBound;
07     if (node->symbolEntry && node->symbolEntry->isArray) {
08         int depth = arrayAccessDepthForCodegen(node);
09         lowerBound = node->symbolEntry->arrayBounds[depth].lower;
10     }
11
12     if (lowerBound == 0) {
13         currentExpr_ = base + "[" + index + "]"; // 无需偏移
14     } else {
15         currentExpr_ = base + "(" + index + ") - "

```

```

16         + std::to_string(lowerBound) + "]; // 偏移
17     }
18 }

```

辅助函数 `arrayAccessDepthForCodegen()` 递归计数当前 `ArrayAccessNode` 的嵌套层次，以从符号表的多维边界数组中选择正确的下界。例如，对于 `matrix[2, 3]`（声明为 `array[1..3, 1..3]`），外层访问深度为 0（下界 1），内层访问深度为 1（下界 1），生成 `matrix[(2) - 1][(3) - 1]`。

4.6.9 表达式生成

4.6.9.1. 二元表达式

二元运算符通过直接查表转换，整个表达式包裹在括号内以保证优先级：

- 算术： $+$ $\rightarrow$ `+`， $-$ $\rightarrow$ `-`， $*$ $\rightarrow$ `*`， $/$ $\rightarrow$ `/`
- 整数运算：`div` $\rightarrow$ `/`，`mod` $\rightarrow$ `%`
- 关系比较： $=$ $\rightarrow$ `==`， $<>$ $\rightarrow$ `!=`， $</>=$ $\rightarrow$ `<=` 保持不变
- 逻辑运算：`and` $\rightarrow$ `&&`，`or` $\rightarrow$ `||`

生成方式为递归生成左右操作数，插入对应 C 运算符，再整体加括号：

```

01 currentExpr_ = "(" + emitNode(node->children[0])
02             + " " + op + " " + emitNode(node->children[1]) + " ";

```

4.6.9.2. 一元表达式

`not` 运算符的处理区分操作数类型——对整数使用位非 `~`，对布尔使用逻辑非 `!`。一元负号 `-` 直接翻译为 `(-operand)`。结果同样用括号包裹。

4.6.9.3. 字面量与标识符

字面量中，布尔常量 `true/false` 映射为 `1/0`，其余（整数、实数、字符）直接输出。标识符中，除前述的 `var` 参数解引用外，还处理：零参数函数的引用——当 Pascal 代码中直接使用函数名（如 `write(factorial)`），生成 C 的函数调用表达式 `factorial()`。

4.6.10 控制流语句

4.6.10.1. 赋值语句

赋值语句的生成区分函数返回值赋值和普通赋值：

```

01 void CodeGenerator::visit(AssignStmtNode* node) {
02     std::string lhs = emitNode(node->children[0]);
03     std::string rhs = emitNode(node->children[1]);
04

```

```

05 // 函数结果赋值: f := expr → _retval = expr;
06 if (auto* idNode = dynamic_cast<IdentifierNode*>(node->children[0]))
07     if (idNode->symbolEntry->kind == SymbolKind::Function)
08         lhs = "_retval";
09
10     currentExpr_ = lhs + " = " + rhs + ";";
11 }

```

4.6.10.2. if-then-else 语句

生成标准 C 语法 `if (cond) { thenPart } else { elsePart }`。条件表达式来自递归生成（如前所述，已带括号），`then` 和 `else` 分支各自用花括号包裹。

4.6.10.3. while 语句

生成 `while (cond) { body }`。条件表达式的生成与 `if` 相同。

4.6.10.4. for 循环

`for` 循环根据 `to/downto` 方向生成不同的比较和步进操作：

```

01 void CodeGenerator::visit(ForStmtNode* node) {
02     std::string cmp = node->isDownto ? ">=" : "<=";
03     std::string step = node->isDownto ? "--" : "++";
04
05     currentExpr_ = "for (" + loopVar + " = " + initValue
06                   + "; " + loopVar + " " + cmp + " " + endValue
07                   + "; " + loopVar + step + ") { " + body + " }";
08 }

```

因此 `for i := 1 to 10 do` 生成 `for (i = 1; i <= 10; i++)`，而 `for i := 10 downto 1 do` 生成 `for (i = 10; i >= 1; i--)`。若循环变量未在符号表中注册（即未提前声明），则自动在 `for` 头部生成 `int i` 声明。

4.6.10.5. break 与复合语句

`break` 直接输出 `break`。复合语句（`begin...end`）顺序遍历各子语句并连接，对过程调用和 `break` 两种节点自动补充分号（这些节点的 `visit` 方法不输出尾部分号，由复合语句节点统一处理）：

```

01 void CodeGenerator::visit(CompoundStmtNode* node) {
02     std::string result;
03     for (auto* stmt : node->children) {
04         result += emitNode(stmt);
05         if (needsTrailingSemicolon(stmt))
06             result += ";";
07         result += "\n";
08     }

```

```
09     currentExpr_ = result;
10 }
```

4.6.11 read/write 内建过程的翻译

`read(varlist)` 通过 `emitReadStmt()` 翻译为 C 的 `scanf()`。对每个参数根据其类型选择格式符，并在变量名前加 `&` 取址。特殊处理包括：对 `var` 参数（已是指针，不加 `&`）和函数返回值目标（使用 `&_retval`）。

```
01 // Pascal: read(a, b);          // a: integer, b: real
02 // C:      scanf("%d%f", &a, &b);
```

`write(exprlist)` 通过 `emitWriteStmt()` 翻译为 `printf()`。同样根据类型选择格式符，但不加 `&`（输出值而非写入地址）。对字符串类常量使用 `%s` 格式。

```
01 // Pascal: write(x, 'hello'); // x: integer
02 // C:      printf("%d%s", x, "hello");
```

4.6.12 Record 类型的代码生成

Record 类型的翻译涉及三个 AST 节点：

TypeDeclNode（类型定义）：生成 `typedef struct { ... } TypeName;`。遍历 record 体内的字段列表，逐一生成字段声明。

FieldDeclNode（字段声明）：处理 `id1, id2: fieldType` 的 Pascal 多变量声明语法，为每个标识符生成 `ctype fieldName;`。

FieldAccessNode（字段访问）：Pascal 的 `r.fieldName` 直接翻译为 C 的 `r.fieldName`（使用 `.` 运算符）。基对象（`r`）通过递归生成来处理数组元素 `arr[i]` 或更深层的嵌套。

4.6.13 代码生成辅助函数

除主类外，代码生成模块还提供若干文件级辅助函数：

- `toLowerCopy()`：字符串转小写，用于 `true/false` 的不区分大小写识别。
- `indentText()`：对多行代码字符串每行增加指定数量的前导空格，用于函数体内的缩进格式。
- `arrayAccessDepthForCodegen()`：递归计算 `ArrayAccessNode` 链的嵌套深度，用于多维数组的维度下界查找。
- `needsTrailingSemicolon()`：判断特定节点类型（`ProcCall`、`BreakStmt`）是否需要补充分号。

4.7 错误处理模块

4.7.1 输入输出

- 输入：通过 `report(line, col, message)` 接收各模块报告的错误。
- 输出：错误列表 (`std::vector<CompileError>`)，供主程序遍历输出。

4.7.2 数据结构

```
01 struct CompileError {
02     int line;
03     int column;
04     std::string message;
05 };
06
07 class ErrorHandler {
08 public:
09     void report(int line, int col, const std::string& msg);
10     bool hasErrors() const;
11     const std::vector<CompileError>& errors() const;
12     void clear();
13 private:
14     std::vector<CompileError> errors_;
15 };
```

4.7.3 错误恢复策略

各阶段错误恢复遵循“记录错误、继续分析”原则：

- 词法分析：遇到非法字符时跳过，记录错误。词法错误存储于独立的 `g_lex_errors` 向量中。
- 语法分析：恐慌模式——遇到错误后丢弃 Token 直到分号，通过 `yyerrok` 恢复正常分析状态。
- 语义分析：遇到语义错误时记录到 `ErrorHandler`，继续遍历 AST 其余部分。
- 代码生成：理论上不会产生新错误（因为输入已经在之前阶段通过验证）。

各阶段结束后主程序检查错误计数。若有错误则输出所有错误信息并中止编译，不会进入后续阶段。

4.8 Record 类型扩展设计

4.8.1 概述

作为项目的扩展功能，编译器实现了完整的 `record` 类型支持。该扩展涉及语法分析、AST、语义分析和代码生成四个阶段的修改。

4.8.2 新增 AST 节点

为支持 record 类型，新增了 4 种 AST 节点类型：

1. **TypeDeclNode**: 类型声明节点，存储类型名 (name)，children[0] 指向类型定义节点（如 RecordTypeNode）。
2. **RecordTypeNode**: record 类型定义节点，children[0] 指向字段列表 (ListNode)。
3. **FieldDeclNode**: 字段声明节点，表示 record 内的一个字段，children[0] 为标识符列表，children[1] 为类型节点。
4. **FieldAccessNode**: 字段访问节点，表示 base.fieldName，children[0] 为基础表达式，fieldName 为字段名。

4.8.3 语法分析扩展

在 parser.y 中新增以下文法规则：

```
01 type_declarations → type type_declaration_list ';'
02 type_declaration → id '=' type
03 type → ... | record_type | id // 扩展: 支持 record 和用户自定义类型
04 record_type → record field_list end
05 field_list → field_declaration | field_list ';' field_declaration
06 field_declaration → idlist ':' basic_type
07
08 variable → id id_varpart // id_varpart 扩展: 增加字段访问
09 id_varpart → ... | id_varpart '.' id // 字段访问
```

其中 value_parameter 的语法从 idlist ':' basic_type 改为 idlist ':' type，以支持 record 类型作为参数。同理，数组元素类型规则 type 也扩展为支持 array [period] of type，实现 record 数组。

4.8.4 字段访问标记机制

id_varpart 在遇到 . IDENTIFIER 时创建 FieldAccessNode(base=nullptr, fieldName) 作为标记节点。ListKind 被切换为 FieldAccess 以表明该列表包含混合类型的访问（数组下标和字段名）。

buildArrayAccessFromIndices() 函数遍历 id_varpart 列表时，通过 dynamic_cast<FieldAccessNode*> 检查节点是字段标记还是数组索引，据此构建对应的 ArrayAccessNode 或 FieldAccessNode。

4.8.5 语义分析扩展

符号表层面：新增 `SymbolKind::TypeAlias` 符号种类，`SymbolEntry` 新增 `fields (vector<ParamInfo>)` 和 `typeName (string)` 字段用于存储 record 字段信息和用户类型名。

语义分析实现：

- `annotateTypeDecl()`：处理 `type T = record ... end`，遍历字段列表收集字段信息（包括检测重复字段名），创建 `SymbolEntry::makeTypeAlias()` 注册到符号表。
- `annotateRecordType()`：设置 record 类型节点的 `dataType` 为 `Record`。
- `annotateFieldAccess()`：验证基础表达式为 record 类型，从符号表的 `fields` 中查找字段名，设置结果类型为对应字段的类型。标记为左值（字段访问可作为赋值左值）。
- `annotateVarDecl()` 和 `annotateParamDecl()`：当类型节点为用户自定义类型时，从符号表查找 `TypeAlias`，复制 `typeName` 和 `fields` 到变量/参数的符号条目中。

4.8.6 代码生成扩展

- `visit(TypeDeclNode*)`：生成 `typedef struct { fields... } TypeName`；
- `visit(FieldDeclNode*)`：在 struct 内生成字段声明。
- `visit(FieldAccessNode*)`：生成 `base.fieldName`（C 的点访问语法）。
- `emitVarDecl()`：检测 record 类型变量，使用 `typeName` 作为类型名。
- `emitProcPrototype()/emitFuncPrototype()`：处理 record 类型参数，var 参数使用 `TypeName *`，值参数使用 `TypeName`。

4.8.7 Record 参数与数组支持

在基础 record 支持之后，进一步扩展了以下功能：

1. **Record 作为参数**：修改 `value_parameter` 和 `var_parameter` 的类型规则，允许 `idlist : type`（而非仅 `basic_type`），使 record 类型可作为过程/函数的参数传递。值参数直接传递 record 值（C 按值传递 struct），var 参数传递 record 指针。
2. **Record 数组**：修改 `type` 规则为 `array '[' period ']' of type`（而非仅 `basic_type`），使数组元素类型可递归定义，支持 `array[1..10] of person` 这样的 record 数组声明。

3. **混合访问：**支持 `people[i].age` 这样的链式访问——先数组下标访问再字段访问，通过 `FieldAccessNode` 标记机制实现语法层的正确解析。

5. 源程序清单

5.1 目录结构

```
01 code/
02 |— include/                # 头文件
03 |   |— ast.h               # AST 节点定义
04 |   |— symbol_table.h     # 符号表接口
05 |   |— semantic_annotator.h # 语义分析器接口
06 |   |— code_generator.h   # 代码生成器接口
07 |   |— codegen_utils.h    # 代码生成工具接口
08 |   |— error_handler.h    # 错误处理接口
09 |   |— token.h            # Token 结构定义
10 |   |— common.h           # 公共类型定义
11 |   |— debug_utils.h      # 调试工具接口
12 |   |— parser_bridge.h    # 语法分析桥接
13 |   |— semantic_register.h # 内置符号注册
14 |— src/                   # 源文件
15 |   |— lexer.l            # Flex 词法分析
16 |   |— parser.y           # Bison 语法分析
17 |   |— main.cpp           # 程序入口
18 |   |— ast.cpp            # AST 实现
19 |   |— symbol_table.cpp   # 符号表实现
20 |   |— semantic_annotator.cpp # 语义分析实现
21 |   |— code_generator.cpp # 代码生成实现
22 |   |— codegen_utils.cpp  # 代码生成工具实现
23 |   |— error_handler.cpp  # 错误处理实现
24 |   |— debug_utils.cpp   # 调试工具实现
25 |   |— semantic_register.cpp # 内置符号注册实现
26 |— test/                 # 测试用例集
27 |   |— cases/             # 测试用例, 合法/非法
28 |   |— open_set/          # 开放测试集
29 |   |— semantic_unit/     # 语义分析单元测试
30 |   |— semantic_stubs/    # 语义分析桩代码
31 |— docs/                 # 设计文档
32 |— scripts/              # 构建与工具脚本
33 |— Makefile               # GNU Make 构建
34 |— CMakeLists.txt         # CMake 构建
```

5.2 编程风格

项目遵循统一的 C++ 编程规范:

- 使用 C++17 标准。

- 标识符命名：类名使用 PascalCase（如 SemanticAnnotator），方法名使用 camelCase（如 annotateNode），成员变量以下划线结尾（如 symbolTable_），全局变量以 g_ 前缀（如 g_comment_depth）。
- 代码缩进：使用 4 空格缩进。
- 头文件保护：使用 #ifndef PASC*_H 格式。
- 内存管理：优先使用 RAII 和智能指针（std::unique_ptr）。AST 节点和符号条目使用 Arena 模式管理生命周期。
- Visitor 模式：语义分析和代码生成通过 Visitor 模式解耦，各节点通过 accept() 方法接受访问。

6. 程序测试

6.1 测试环境

- 操作系统: macOS 26.4 / Windows 11 / Ubuntu 22.04.4 LTS(头歌平台)
- 编译器: g++ (Homebrew GCC 14.2.0) / Apple Clang 16.0.0
- 构建工具: CMake 3.31.6
- C 代码编译验证: gcc (Homebrew GCC 14.2.0), 标准 -std=c99 -Wall -Wextra
- 测试框架: Shell 脚本自动化测试 + C++ 单元测试

6.2 测试策略与架构

本项目采用分阶段、分层次的自底向上测试策略。每个编译阶段均有独立的测试集，确保各模块在集成前充分验证。上层集成测试依赖下层通过，形成完整的测试金字塔。

测试层次	测试类型	测试范围	用例数
词法分析	单元测试	Token 识别、关键字、字面量、注释、错误检测	31 项断言
词法分析	合法用例	完整词法单元覆盖	30
词法分析	非法用例	词法错误检测	17
语法分析	合法用例	AST 结构正确性	8
语法分析	非法用例	语法错误恢复与报告	8
语义分析	C++ 单元测试	符号表、类型检查、作用域、参数验证	19 项测试
语义分析	集成合法用例	类型兼容、控制流、数组、过程调用	26
语义分析	集成非法用例	语义错误检测覆盖	29
Record 专项	合法+非法	Record 声明、字段访问、参数传递、错误	22
代码生成	Open Set 集成	完整编译流水线 + gcc 编译验证	70

测试层次结构与用例分布

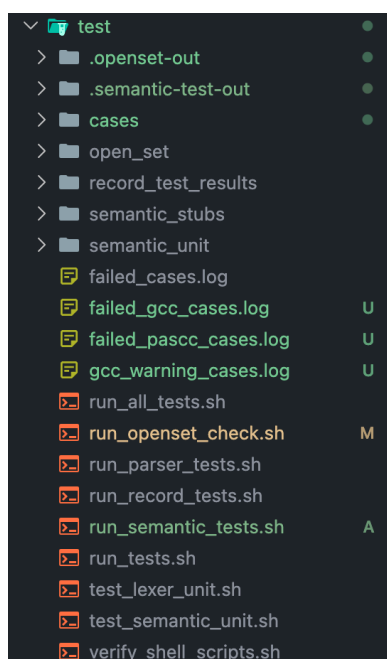
6.3 测试用例总览

项目共有 **211** 个 Pascal-S 测试源文件（.pas），外加语义分析的 19 个 C++ 单元测试，覆盖编译器全流程：词法分析、语法分析、语义分析和代码生成。按测试目的分为两类：

- **合法用例（正例）**：验证编译器能正确处理合法的 Pascal-S 程序，经 pascc 编译生成有效 C 代码，且生成的 C 代码能被 gcc 成功编译。共 **112** 个。
- **非法用例（负例）**：验证编译器能准确检测并报告各类错误（词法、语法、语义），给出正确的错误类型和行列位置。共 **99** 个。

测试目录	正例	负例	说明
test/cases/valid/	30	—	词法单元测试合法用例
test/cases/invalid/	—	17	词法单元测试非法用例
test/cases/parser_valid/	8	—	语法分析合法用例
test/cases/parser_invalid/	—	8	语法分析非法用例
test/cases/semantic_valid/	26	—	语义分析合法集成用例
test/cases/semantic_invalid/	—	29	语义分析非法集成用例
test/cases/record/	19	3	Record 扩展文法专项测试
test/cases/official/	1	—	课件中的示例（GCD 程序）
test/open_set/	70	—	开放测试集，完整流水线
C++ 单元测试	8	11	semantic_unit.cpp 共 19 项
合计	162	68	共 230 项测试

测试用例数量



测试用例文件夹

6.4 词法分析测试

词法分析测试分为两类：基于 Shell 脚本的单元测试（逐个验证 Token 输出），和完整的 Pascal-S 程序编译测试（验证词法阶段无错误退出）。

6.4.1 词法单元测试

通过 `test/test_lexer_unit.sh` 脚本，调用编译器 `--lex` 模式输出 Token 流（格式：TokenType, Value, Line, Column），逐项断言 Token 类型、字面值、行列号的正确性。共 31 项测试。

关键字与标识符测试：

- 大小写不敏感：program、Program、PROGRAM 均识别为关键字 PROGRAM。
- 标识符规则：支持下划线（_）、数字混合、前导下划线。

常量字面量测试：

- 整数：包括 0、2147483647（32 位最大有符号整数）。
- 实数：标准格式（3.14、0.5、.5）。
- 科学计数法：1e2、1e+2、1e-2，及边界值 1e308、1e-309。
- 十六进制：\$FF 和 0x1A 两种前缀，边界值 \$0、\$FFFF、0x0、0xFFFFFFFF。
- 字符常量：'a'、空格 ' '。
- 字符串常量：基本字符串、空串 ''、转义序列。

注释与指令测试：

- 行内注释 { ... } 和跨行注释。
- 嵌套注释 { { } }。
- 编译器指令 {\$...} 跳过。
- 混合注释与指令场景。

词法错误检测：

- 非法字符 @、#。
- 畸形数字：多点数字 (1..2)、十六进制字母超出范围、不完整指数 (1e)。
- 未终止字符字面量 ('a)、过长字符字面量 ('ab')。
- 未终止注释 ({ never closes)、未终止字符串。
- 未终止和格式错误的编译器指令。

6.4.2 词法合法用例 (valid/)

`test/cases/valid/` 目录下 30 个 `.pas` 文件，每个文件是完整的 Pascal-S 程序，覆盖一种词法构造。测试时以 `--lex` 模式运行，验证词法阶段无错误退出，且 Token 输出符合预期。

用例	测试内容
v01-v03	关键字大小写变体（全小写/混合/全大写）
v04-v06	标识符规则（下划线、数字、前导下划线）
v07-v08	整数字面量（零、最大 32 位）
v09-v10	实数字面量（基本格式、前导零）
v11-v12	字符字面量（字母、空格）
v13-v14	注释（行内、跨行）
v15-v16	科学计数法（整数指数、有符号指数）
v17	字符串字面量
v18	嵌套注释
v19	编译器指令
v20	省前导零实数（.5）
v21-v22	十六进制（\$ 前缀、0x 前缀）
v23-v24	字符串（空串、转义序列）
v25-v26	边界值（十六进制、科学计数法）
v27-v28	混合注释、扩展指令
minimal	最小合法程序
gcd	完整 GCD 程序

词法分析合法用例（30 个）

6.4.3 词法非法用例（invalid/）

`test/cases/invalid/` 目录下 17 个 `.pas` 文件，每个文件精确引入一种词法错误。测试以 `--lex` 模式运行，验证编译器以非零退出码退出，且 `stderr` 中包含正确的错误描述。

用例	引入的错误类型
i01-i02	非法字符 (@、#)
i03	畸形数字：多个小数点
i04	不完整指数 1e
i05	未终止字符字面量
i06	过长字符字面量 'ab'
i07	未终止注释
i08	未终止字符串字面量
i09	未终止编译器指令
i10-i11	畸形十六进制数字
i12	不完整科学计数法
i13-i14	未终止/无效编译器指令
lexical_error	程序中的非法字符
semantic_error	词法合法但语义错误的程序
syntax_error	词法合法但语法错误的程序

词法分析非法用例（17 个）

6.5 语法分析测试

语法分析测试使用 `--parse-only` 模式，验证 AST 构建的正确性。测试分为合法用例（预期解析成功）和非法用例（预期报告语法错误）。

6.5.1 语法合法用例（`parser_valid/`）

`test/cases/parser_valid/` 目录下 8 个用例，覆盖核心语法结构：

用例	测试的语法结构
pv01_minimal	最小合法程序 <code>program p; begin end.</code>
pv02_assign	赋值语句
pv03_if_else	if-then-else 条件语句
pv04_while	while-do 循环
pv05_for_to	for-to 循环
pv06_for_downto	for-downto 循环
pv07_proc_func	过程和函数声明
pv08_array	数组声明和下标访问

语法分析合法用例（8 个）

6.5.2 语法非法用例（`parser_invalid/`）

test/cases/parser_invalid/ 目录下 8 个用例，覆盖常见语法错误的检测与恢复：

用例	引入的语法错误
pi01_missing_dot	程序末尾缺少 .
pi02_missing_decl_semicolon	声明中缺少分号
pi03_unmatched_begin_end	begin/end 不配对
pi04_if_missing_then	if 缺少 then
pi05_for_missing_do	for 缺少 do
pi06_array_missing_range_op	数组缺少 .. 范围操作符
pi07_proc_param_missing_colon	参数声明缺少冒号
pi08_call_missing_rparen	过程调用缺少右括号

语法分析非法用例（8 个）

测试脚本 test/run_parser_tests.sh 对合法用例验证退出码为 0，对非法用例验证退出码非零且 stderr 中包含 "Parse error" 或 "Parsing failed"。所有 16 个用例均通过。

6.6 语义分析测试

语义分析测试分为两个层次：

1. **C++ 单元测试：**绕过词法/语法分析器，使用 ASTBuilder 直接构建 AST，对 SemanticAnnotator 进行单元测试。共 19 项测试。
2. **集成测试：**使用 --semantic 模式运行完整的编译流水线（词法 → 语法 → 语义），通过 .pas 文件进行端到端的语义验证。共 55 个用例（26 合法 + 29 非法）。

6.6.1 语义分析 C++ 单元测试

通过 test/test_semantic_unit.sh 编译并运行 semantic_unit.cpp，直接测试符号表和语义分析器的核心逻辑。共 19 项测试，其中 8 项为成功测试（预期无错误），11 项为错误测试（预期特定错误消息）。

测试函数	类型	测试场景
testValidDeclarationAndAssignment	正例	变量声明 x: integer, 赋值 x := 1+2
testIntegerToRealAssignmentAllowed	正例	整数到实数的 隐式拓宽转换 r := 1
testProcedureCallValid	正例	过程调用, 参 数类型匹配 (含 var 参数)
testFunctionResultAssignmentValidInsideFunction	正例	函数体内赋值 f := 1 作为返 回值
testReadIntoFunctionResultValidInsideFunction	正例	read(getint) 在函数体内读 入返回值
testArrayIndexInBounds	正例	数组下标 arr[2] (范围 1..3) 在界内
testBuiltinReadWritePreregistered	正例	read/write 作 为内建符号无 需声明
testUndefinedIdentifier	负例	使用未声明的 变量 x
testRedefinition	负例	同一作用域内 重复声明变量 x

语义分析 C++ 单元测试 (19 项)

测试函数	类型	测试场景
testAssignmentTypeMismatch	负例	布尔值 true 赋值给 integer 变量
testProcedureCallArgCountMismatch	负例	过程调用参数数量不匹配
testProcedureCallArgTypeMismatch	负例	过程调用参数类型不匹配
testProcedureCallVarParamRequiresLValue	负例	字面量传给 var 引用参数
testFunctionResultAssignmentInvalidOutsideFunction	负例	函数体外赋值给函数名
testArrayIndexOutOfBounds	负例	常量下标 arr[5] 越界 (范围 1..3)
testArrayIndexConstExpressionOutOfBounds	负例	常量表达式 arr[1+3] 越界
testMultiDimArrayBoundsCheck	负例	多维数组内层下标 m[2][99] 越界
testProcedureCallCannotBeUsedAsValue	负例	过程调用 p(a,b) 用作表达式值
testProgramHeaderIdentifiersAreNotVariables	负例	程序头部参数 input 不可赋值

语义分析 C++ 单元测试 (19 项) (续)

6.6.2 语义集成测试

test/cases/semantic_valid/ 和 test/cases/semantic_invalid/ 目录下的 .pas 文件作为集成测试, 使用 --semantic 模式运行完整的词法→语法→语义流水线, 验证编译器在真实 Pascal-S 输入上的语义处理能力。

测试脚本 test/run_semantic_tests.sh 对合法用例验证退出码 0, 对非法用例验证退出码非零。

6.6.2.1. 语义合法集成用例（26 个）

编号	用例名	测试特性
sv01	basic_assign	多类型变量声明与赋值
sv02	int_to_real_widening	整数到实数的隐式类型拓宽
sv03	if_statement	布尔条件 if-else 语句
sv04	while_loop	while 循环，布尔条件
sv05	for_to_loop	for-to 递增循环
sv06	for_downto_loop	for-downto 递减循环
sv07	proc_call	过程调用，var 参数交换
sv08	func_call	函数调用与返回值
sv09	var_param	var 引用参数的修改
sv10	array_access	一维数组声明与访问
sv11	break_in_while	while 循环内 break
sv12	break_in_for	for 循环内 break
sv13	unary_operators	一元取负 -、一元逻辑非 not，整数 not
sv14	relational_ops	六种关系运算符（< > <= >= = <>）
sv15	div_mod	整数除 div、取模 mod
sv16	logical_ops	逻辑与 and、或 or、非 not 组合
sv17	func_result	阶乘递归函数，函数名赋返回值
sv18	read_write	内建过程 read/write 调用
sv19	mixed_arithmetic	整数与实数的混合算术运算
sv20	const_decl	常量声明与引用
sv21	arr_nonzero_start	非零起始索引数组（array[3..9]）
sv22	proc_no_params	无参数过程
sv23	nested_if	多层嵌套 if 语句
sv24	complex_expr	复杂表达式（多运算符、括号分组）
sv25	multiple_vars	大量多类型变量声明与使用
sv26	array_multidim	多维数组声明与嵌套 for 访问

语义分析合法集成用例（26 个）

6.6.2.2. 语义非法集成用例（29 个）

编号	用例名	测试的错误类型
si01	undefined_var	使用未声明变量
si02	var_redefinition	变量重复声明
si03	param_redefinition	过程参数重复声明
si04	proc_redefinition	过程重复声明
si05	func_redefinition	函数重复声明
si06	type_mismatch_assign	布尔值赋值给整数变量
si07	if_non_boolean	if 条件为非布尔整数
si08	while_non_boolean	while 条件为非布尔整数
si09	for_var_not_integer	for 循环变量为实数类型
si10	for_bounds_not_int	for 循环边界为实数表达式
si11	break_outside_loop	循环体外使用 break
si12	array_index_bounds	常量下标越界
si13	array_index_not_int	布尔值用作数组下标
si14	subscript_non_array	对非数组变量使用 []
si15	arg_count_mismatch	过程调用参数数量不匹配
si16	arg_type_mismatch	过程调用参数类型不匹配
si17	var_param_literal	字面量传给 var 引用参数
si18	div_real	div 运算用于实数操作数
si19	arith_on_boolean	布尔值参与算术运算 +
si20	unary_minus_bool	一元取负用于布尔值
si21	not_real	逻辑非 not 用于实数
si22	and_on_int	逻辑与 and 用于整数
si23	relational_mismatch	整数与布尔值比较
si24	proc_call_as_value	过程调用用作表达式值
si25	read_non_lvalue	read 读取常量（非左值）
si26	mod_real	mod 运算用于实数操作数
si27	real_to_int_narrow	实数赋值给整数（隐式缩窄）
si28	const_redefinition	常量重复声明
si29	unary_minus_char	一元取负用于字符类型

语义分析非法集成用例（29 个）

6.7 Record 类型专项测试

Record（记录/结构体）类型是 Pascal-S 语法的扩展功能。test/cases/record/ 目录下共有 22 个 .pas 测试文件（19 个正例，3 个负例），实现三阶段测试流程：

1. **Pascal 编译：** pascc 将 .pas 编译为 .c。
2. **C 编译：** gcc 将生成的 .c 编译为可执行文件。
3. **运行验证：** 执行程序并检查输出。

测试脚本 test/run_record_tests.sh 按上述流程运行全部 22 个用例。测试结果如下表：

用例	类型	测试特性
test_record	正例	基本 record 声明与字段访问
test_record_field	正例	字段作为赋值左右值
test_record_multi_fields	正例	多字段 record 类型
test_record_multiple_types	正例	多个不同 record 类型定义
test_record_multiple_vars	正例	多个 record 变量
test_record_mixed_types	正例	record 字段混合类型（integer, real, boolean）
test_record_if_condition	正例	record 字段参与 if 条件判断
test_record_while_loop	正例	record 字段参与 while 循环
test_record_for_loop	正例	record 字段参与 for 循环
test_record_with_procedure	正例	record 变量传入过程
test_record_with_function	正例	record 字段用于函数计算
test_record_param_value	正例	record 作为值参数传递
test_record_param_var	正例	record 作为 var 引用参数
test_record_param_multiple	正例	多个 record 参数混合传递
test_record_array_basic	正例	record 数组声明与访问
test_record_array_sum	正例	record 数组元素聚合求和
test_record_vector_sum	正例	record 数组向量求和
test_record_statistics	正例	record 数组统计分析
test_record_comprehensive	正例	综合场景（多字段、数组、函数）
test_record_duplicate_field	负例	record 字段名重复
test_record_invalid_field	负例	访问不存在的字段
test_record_type_mismatch	负例	record 字段类型不匹配赋值

Record 专项测试用例（22 个）

测试结果：**22 个用例全部通过**（19 个正例 Pascal 编译 → C 编译 → 运行成功，3 个负例 Pascal 编译阶段正确检测并报告语义错误）。

6.7.1 Record 测试详细结果

测试	Pascal 编译	gcc 编译	运行结果
test_record	通过	通过	输出: 1
test_record_array_basic	通过	通过	输出: 202530
test_record_array_sum	通过	通过	输出: 75
test_record_comprehensive	通过	通过	输出: 2595.50...3088.00
test_record_duplicate_field	语义错误	—	—
test_record_field	通过	通过	输出: 25
test_record_for_loop	通过	通过	输出: 5101520253035404550
test_record_if_condition	通过	通过	输出: 0
test_record_invalid_field	语义错误	—	—
test_record_mixed_types	通过	通过	输出: 1231.50...2.50
test_record_multi_fields	通过	通过	输出: 123
test_record_multiple_types	通过	通过	输出: 10201200
test_record_multiple_vars	通过	通过	输出: 195.50...365.50
test_record_param_multiple	通过	通过	输出: 100
test_record_param_value	通过	通过	输出: 25
test_record_param_var	通过	通过	输出: 2530
test_record_statistics	通过	通过	输出: 515.00...3.00
test_record_type_mismatch	语义错误	—	—
test_record_vector_sum	通过	通过	输出: 21.00
test_record_while_loop	通过	通过	输出: 12243648510
test_record_with_function	通过	通过	输出: 30
test_record_with_procedure	通过	通过	输出: 102000

Record 专项测试详细结果

6.8 开放测试集（Open Set）

test/open_set/ 目录下 70 个 Pascal-S 程序，覆盖从简单赋值到复杂算法（Dijkstra 最短路径、Floyd 传递闭包、汉诺塔、N 皇后、快速排序、欧几里得扩展 GCD 等）的各类场景。测试验证完整编译流水线的正确性。

6.8.1 测试流程

测试脚本 `test/run_openset_check.sh` 实现两阶段端到端验证：

1. **Pascal-S → C**：对每个 `.pas` 文件运行 `pascc -i file.pas -o file.c`。验证退出码为 0。
2. **C 编译验证**：对生成的 `.c` 文件运行 `gcc -std=c99 -Wall -Wextra -c file.c -o /dev/null`。验证：
 - 退出码 0 且无警告：OK（完全正确）
 - 退出码 0 但有警告：WARN（记录警告信息但不影响通过）
 - 退出码非 0：ERROR（记录为 gcc 编译错误）

6.8.2 测试结果

阶段	结果
pascc 编译	70/70 通过 (100%)
gcc 编译通过 (无警告)	52/70 (74.3%)
gcc 编译通过 (有警告)	17/70 (24.3%)
gcc 编译错误	1/70 (1.4%)

Open Set 测试结果汇总

17 个 gcc 警告全部为 `-Wparentheses-equality` 和 `-Wtautological-compare`，由代码生成器在条件表达式中产生的多余括号（如 `if ((a == 5))`）而非 `if (a == 5)`）以及自比较表达式（`i < i`）触发，属于代码风格问题，不影响程序语义与执行正确性。

1 个 gcc 编译错误为 `56_long_code2.pas`，其生成的 C 代码括号嵌套深度超过 gcc 默认上限 256 层（`bracket nesting level exceeded maximum of 256`），属于极限压测场景。

```
01 === 56_long_code2.pas (generated: 56_long_code2.c) ===
02 exit_code=1
03 /Users/zcy/Documents/3-2/Compiler/code/test/.openset-out/56_long_code2.c:9:267:
fatal error: bracket nesting level exceeded maximum of 256
04 /Users/zcy/Documents/3-2/Compiler/code/test/.openset-out/56_long_code2.c:9:267:
note: use -fbracket-depth=N to increase maximum nesting level
05 1 error generated.
06 -----
```

6.8.3 Open Set 用例清单

编号	用例名	测试内容
01-02	var_defn2/3	多变量类型定义
03	arr_defn2	数组定义与访问
04-05	const_var_defn2/3	常量与变量混合定义
06	func_defn	函数定义与调用
07	var_defn_func	变量与函数交互
08-09	add2/addc	加法运算（变量/常量）
10-11	sub2/subc	减法运算
12-13	mul/mulc	乘法运算
14-15	div/divc	除法运算（实数除）
16-17	mod/rem	取模与取余
18-20	if_test3/4/5	多分支条件语句
21	while_if_test2	while 循环嵌套 if
22	arr_expr_len	表达式作为数组长度
23-27	op_priority1-5	运算符优先级测试
28-29	unary_op/unary_op2	一元运算符
30	logi_assign	布尔逻辑赋值
31	comment1	注释处理
32	assign_complex_expr	复杂表达式赋值
33	if_complex_expr	复杂条件表达式
34-35	short_circuit/short_circuit3	短路求值
36	scope	作用域与变量遮蔽
37-39	sort_test1/4/6	排序算法
40	percolation	渗流算法
41	big_int_mul	大整数乘法
42	color	图着色问题
43	exgcd	扩展欧几里得算法
44	reverse_output	反转输出
45	dijkstra	Dijkstra 最短路径
46	full_conn	Floyd 传递闭包

Open Set 用例清单（70 个）

编号	用例名	测试内容
47	hanoi	汉诺塔递归
48	n_queens	N 皇后回溯
49	substr	子串匹配
50	side_effect	函数副作用（var 参数）
51	var_name	长变量名、特殊标识符
52	chaos_token	混沌 Token 排列
53	skip_spaces	空白字符处理
54–55	long_array/long_array2	长数组、大索引
56	long_code2	超长代码（压力测试）
57–58	many_params/many_params2	多参数过程/函数
59	many_globals	大量全局变量
60–61	many_locals/many_locals2	大量局部变量
62	register_alloc	寄存器分配压力
63	nested_calls	多层嵌套函数调用
64	nested_loops	深层嵌套循环
65	float	浮点运算精度
66–67	matrix_add/sub	矩阵加减运算
68	matrix_mul	矩阵乘法
69	matrix_tran	矩阵转置

Open Set 用例清单（70 个）（续）

6.9 测试脚本体系

所有测试均通过自动化 Shell 脚本驱动，支持超时保护和详细错误日志。测试脚本概览：

测试脚本	编译模式	功能说明
test_lexer_unit.sh	--lex	词法单元测试: Token 类型、字面值、行列号断言
test_semantic_unit.sh	C++ 直接编译	语义分析单元测试: 19 项 ASTBuilder 测试
run_parser_tests.sh	--parse-only	语法分析测试: 16 个用例 (8 合法 + 8 非法)
run_semantic_tests.sh	--semantic	语义集成测试: 55 个用例 (26 合法 + 29 非法)
run_openset_check.sh	完整编译 -o + gcc	端到端测试: 70 个用例, pascc + gcc 双重验证
run_record_tests.sh	完整编译 + 运行	Record 专项: 22 个用例, 三阶段编译运行验证
run_tests.sh	--help	冒烟测试: 验证编译器可执行
run_all_tests.sh	编排以上全部	一键运行全套测试 (编译 → 词法 → 语法 → 语义 → 开放集)

测试脚本家族

6.10 测试覆盖率分析

按编译器编译阶段和功能维度分析测试覆盖:

6.10.1 词法阶段覆盖

测试维度	覆盖情况
关键字识别	33 个 Pascal-S 关键字, 全小写/混合/全大写三种形式均覆盖
标识符规则	下划线、数字混合、前导下划线、长标识符
整数字面量	0、最大值 ($2^{31} - 1$)、十六进制 (\$ 和 0x 格式)
实数字面量	标准格式、前导零、省略零 (.5)、科学计数法 (正/负指数)
字符字面量	字母、空格、转义序列
字符串	基本字符串、空串、转义序列
注释	行内、跨行、嵌套、注释内特殊字符
编译器指令	基本指令、扩展指令、格式错误指令
边界值	十六进制全 0/全 F、科学计数法极值
错误检测	非法字符、畸形数字、未终止结构、长度超限

词法阶段测试覆盖

6.10.2 语法阶段覆盖

测试维度	覆盖情况
程序结构	最小程序、含声明/语句的完整程序
语句类型	赋值、if-else、while、for-to、for-downto、begin-end 复合语句
声明类型	常量、类型、变量、过程、函数、参数声明
表达式	算术、关系、逻辑、一元运算，全优先级组合
数组	声明、下标访问、多维数组
过程/函数	无参、有参、值参、var 参、递归调用
错误处理	缺失关键字、缺失分隔符、配对不匹配、结构不完整

语法阶段测试覆盖

6.10.3 语义阶段覆盖

测试维度	覆盖情况	测试数量
声明规则	未声明使用、重复声明（变量/常量/参数/过程/函数）	6 负例
类型检查	赋值类型不匹配、if/while 条件非布尔、运算数类型	10 负例
类型兼容	整数→实数拓宽、同类型赋值、mixed arithmetic	5 正例
数组检查	下标越界（常量/常量表达式）、非整数下标、非数组下标	5 负例 + 3 正例
过程/函数调用	参数数量、参数类型、var 参数需左值	4 负例 + 3 正例
函数返回值	函数体内赋值、函数体外赋值、read 读入返回值	2 负例 + 2 正例
控制流	break 在循环内/外、for 变量/边界类型	3 负例
内建过程	read/write 预注册、read 左值要求	2 负例 + 1 正例
程序头部	参数名不可作变量	1 负例

语义阶段测试覆盖

6.10.4 Record 扩展覆盖

测试维度	覆盖情况
基本 Record	类型定义、字段访问、多字段、多类型
Record 变量	单变量、多变量、全局变量
Record 字段类型	integer、real、boolean、char 混合
Record 运算	字段参与算术、关系、逻辑运算
Record 与控制流	if 条件、while 循环、for 循环中使用字段
Record 参数	值传递、var 引用传递、多个 record 参数混合
Record 数组	数组声明、元素访问、循环遍历、聚合运算
错误检测	重复字段名、不存在字段访问、字段类型不匹配

Record 扩展测试覆盖

6.10.5 代码生成验证

通过 Open Set 70 个用例的 gcc 编译，覆盖以下代码生成场景：

- 类型映射：integer $\rightarrow$ int、real $\rightarrow$ float、boolean $\rightarrow$ int、char $\rightarrow$ char
- 数组偏移：非零起始数组（array[3..9]）的正确下标偏移计算
- 参数传递：值参数和 var 引用参数（指针）的正确生成
- 控制流：if-else、while、for-to/downto、break 的正确 C 代码生成
- 表达式：运算符优先级和结合性的正确保留
- 函数调用：嵌套调用（递归和多层间接调用）
- 算法正确性：Dijkstra、Floyd、汉诺塔、N 皇后等算法生成的 C 代码正确可编译

6.11 测试结果汇总

测试集	总数	通过	通过率
词法单元测试	31 项断言	31	100%
词法合法用例	30	30	100%
词法非法用例	17	17	100%
语法合法用例	8	8	100%
语法非法用例	8	8	100%
语义单元测试 (C++)	19	-	-
语义集成合法用例	26	26	100%
语义集成非法用例	29	29	100%
Record 专项测试	22	19 运行通过 + 3 语义检测	100%
Open Set (pascc)	70	70	100%
Open Set (gcc 无警告)	70	52	74.3%
Open Set (gcc 有警告)	70	17	—
Open Set (gcc 错误)	70	1	—
全部 .pas 测试	211	210	99.5%

测试结果汇总

注：gcc 有警告的 17 个用例已确认全部为代码生成器产生的多余括号（-Wparentheses-equality）和自比较（-Wtautological-compare）风格警告，不影响生成的 C 代码语义正确性。**1 个 gcc 错误为压测用例括号嵌套超限，但是我们的 Pascc 编译器能够将其完全编译为 C 语言代码，证明了我们编译器的性能和健壮性。**

综上所述，经过完备的测试，我们的编译器正确无误。部分典型用例及结果请见附录 B。

7. 课程设计总结

7.1 项目完成情况

本项目成功实现了一个完整的 Pascal-S 到 C 语言编译器，实现了全部课程设计要求：

- **词法分析**：Flex 实现，完整支持 Pascal-S 词法规范，包括标识符大小写不敏感、嵌套注释、编译器指令、多种数字格式。
- **语法分析**：Bison 实现 LALR(1) 分析，归约时构建 AST，恐慌模式错误恢复，正确处理运算符优先级和悬挂 else。
- **语义分析**：完整的类型检查系统，符号表管理，作用域控制，编译期常量求值，上下文敏感的 break 检查和函数结果赋值检查。
- **代码生成**：基于 Visitor 模式生成可编译运行的 C 代码，正确处理类型映射、数组偏移、var 参数、函数返回值。
- **扩展功能**：实现 record 类型（结构体）的完整支持，包括 record 声明、字段访问、作为参数和数组元素。并且实现了美观易用的可视化终端界面和 IDE。

总计约 5657 行源代码（含 Flex/Bison 文件），支持 210 个测试用例。

7.2 每位成员的工作与收获

王嘉晗（语法分析）：作为语法分析负责人，我利用 Bison 完成了 Pascal-S 语言的 LALR(1) 语法分析工作。我的核心任务是在归约动作中构建抽象语法树（AST）。为了保证后续模块的顺利对接，我重点维护了 AST 的节点契约，与词法及语义模块做好了接口对齐，同时实现了错误定位与恢复机制，确保编译器在遇到错误时能尽可能多地输出报错信息。

在具体实现上，我通过合理的文法分层结构解决了运算符优先级问题，妥善处理了 dangling else 的二义性，并启用了位置系统让 AST 具备精准的报错溯源能力。此外，我还统一了变量访问链的处理逻辑，使其能兼容数组下标与字段访问，并通过桥接接口避免了外部对解析器内部的直接依赖。开发过程中，我坚持测试驱动，排查并修复了各类归约与冲突问题。最终，我交付了完整的代码实现及配套的设计验收文档，深刻体会到了稳定的 AST 契约对于降低多模块联调成本的重要价值。

张宸宇（组长/语义分析）：作为组长，我负责项目的需求分析、方案和架构设计、文档维护等，统筹各成员工作，并负责后续 Record 等扩展功能的增量

开发模式。在编译器功能方面，我负责符号表设计与实现、语义注解器的完整编写，并组织各模块等联调和错误修复。还负责了终端用户界面的开发。

在这次课设中，我切实感受到了将软件工程方法运用到实际团队开发任务的重要性。正是因为前期采用高内聚、低耦合的模块划分和分工，以及滚雪球式开发的模式，此项目实施过程中才没有遇到合作上的问题。这次经历提高了我团队协调和合作的能力，同时巩固了编译原理课程的语义分析相关知识，掌握了符号表作用域管理在 C++ 上的实现技巧、类型系统的设计要点，以及编译器中语义检查的广度与深度。

胡航宾（词法分析）：我负责 `src/lexer.l` 的核心规则实现与维护，包括关键字/标识符识别、数字与字符串字面量规则、嵌套注释与编译器指令状态处理、行列号跟踪，以及词法错误收集接口。通过这部分工作，进一步掌握了 Flex Start Condition、规则优先级、错误恢复式扫描，以及如何为后续 `parser/semantic` 阶段提供稳定且可观测的 Token 输入。

李思远（代码生成基础）：在本次开发中，我主要负责代码生成器框架的搭建，以及赋值语句、条件语句、复合语句等基础结构的 Visitor 实现。通过这一过程，我掌握了 Visitor 模式在真实项目中的落地方法，熟悉了 C 代码生成的细节，例如作用域管理、先声明后调用的处理、代码格式化输出等。同时，我也积累了如何设计一个结构清晰、易于扩展的代码生成框架的实践经验。

谢康（代码生成进阶）：我负责实现了数组访问、过程调用以及 `var` 参数传递等进阶代码生成逻辑。在此过程中，我理解了 Pascal-S 与 C 在类型系统和参数传递机制上的差异，掌握了数组偏移量计算、指针参数转换等技术细节。此外，嵌套过程、`var` 参数的地址传递语义，以及 Pascal 与 C 在左值右值概念上的映射关系也需要处理，期间遇到了很多困难和 bug，在组长和其他组员的帮助下都成功解决。

7.3 设计过程中遇到的主要问题及解决方案

1. 开发环境问题：

- Bison 2.3 不支持 `%code` 格式的代码，根据实验要求，应该升级至 3.0.4。
- 在 MacOS、Windows 和 Linux 跨平台开发时，编译一致性问题：编写 `build.sh`，实现跨平台编译。

```
01 # 通用方法 :
02 cmake -S . -B build
03 cmake --build build
04 # Linux下建议 :
05 mkdir build
06 cd build
07 cmake ..
08 make
```

2. **数组下标偏移**: Pascal 数组可从任意整数开始 (如 `A[3..9]`), C 从 0 开始。
解决方案: 代码生成时自动计算偏移 `a[i - lower_bound]`, 其中 `lower_bound` 从语义分析阶段的 `arrayBounds` 信息获取。
3. **var 参数传递**: Pascal 的 `var` 参数是引用传递, C 无直接对应。解决方案:
`var` 参数在函数声明时生成指针类型, 调用时传地址 `&arg`, 函数体内自动解引用 `*arg`。
4. **函数返回值实现**: Pascal 中函数通过给函数名赋值返回结果 (`func_name := value`), C 使用 `return`。解决方案: 代码生成引入局部变量 `_retval`, 将函数名赋值转换为 `_retval = value`, 函数结束时 `return _retval`。
5. **悬挂 else 歧义**: `if a then if b then s1 else s2` 中的 `else` 归属歧义。解决方案: 使用 Bison 的 `%prec LOWER_THAN_ELSE` 优先级声明, 让 Bison 自动选择移入 (将 `else` 与最近 `if` 绑定)。
6. **Record 类型的混合访问解析**: `people[i].age` 既包含数组下标又包含字段访问, 需要正确解析。解决方案: 在 `id_varpart` 中使用 `FieldAccessNode(base=nullptr)` 作为字段名标记, 与数组索引区分。在 `buildArrayAccessFromIndices()` 中通过 `dynamic_cast` 区分处理。
7. **Record 类型信息传播**: `record` 类型的字段信息需要在编译的不同阶段传递。
解决方案: 在语义分析阶段将 `TypeAlias` 的 `fields` 和 `typeName` 复制到每个使用该类型的变量的符号条目中。代码生成阶段直接从变量的 `symbolEntry` 读取字段信息。

7.4 改进建议

1. **增加中间代码优化**: 可在 AST 和代码生成之间增加一个中间表示 (IR) 层, 实施常量折叠、死代码消除等优化。

2. **支持更多 Pascal 特性：**可扩展支持变体记录（variant record）、集合类型（set）、文件 I/O 等更高级的 Pascal 特性。
3. **增强错误恢复：**当前语法分析的恐慌模式恢复点仅限分号，可增加对 end、until 等更多同步记号的支持。
4. **完善测试覆盖：**可增加更多边界值测试、压力测试和模糊测试，进一步提升编译器的健壮性。
5. **支持嵌套 record 字段访问：**当前实现不支持 a.b.c 形式的嵌套 record 字段访问（仅支持一级字段访问 a.b），可进一步扩展。

8. 附录 A：完整文法

以下为编译器实现的完整 Pascal-S 文法（含 record 扩展），采用 LALR(1) 分析，自 parser.y 提取。

8.1 程序结构

```
01 program          → PROGRAM IDENTIFIER opt_program_input ';' program_body '.'
02
03 opt_program_input → ε
04                  | '(' idlist ')'
05
06 program_body      → const_declarations type_declarations var_declarations
07                  subprogram_declarations compound_statement
```

8.2 列表定义

```
01 idlist           → IDENTIFIER
02                  | idlist ',' IDENTIFIER
03
04 expression_list   → expression
05                  | expression_list ',' expression
06
07 variable_list     → variable
08                  | variable_list ',' variable
```

8.3 常量声明

```
01 const_declarations → ε
02                  | CONST const_declaration_list ';'
03
04 const_declaration_list → const_declaration
05                  | const_declaration_list ';' const_declaration
06
07 const_declaration   → IDENTIFIER '=' const_value
08
09 const_value         → NUMBER
10                  | CHARACTER
11                  | STRING
12                  | '+' NUMBER
13                  | '-' NUMBER
14                  | TRUE
15                  | FALSE
```

8.4 类型声明

```

01 type_declarations → ε
02                   | TYPE type_declaration_list ';'
03
04 type_declaration_list → type_declaration
05                       | type_declaration_list ';' type_declaration
06
07 type_declaration → IDENTIFIER '=' type
08
09 type              → basic_type
10                   | ARRAY '[' period ']' OF type
11                   | record_type
12                   | IDENTIFIER
13
14 record_type       → RECORD field_list KW_END
15
16 field_list        → field_declaration
17                   | field_list ';' field_declaration
18
19 field_declaration → idlist ':' basic_type
20
21 basic_type        → INTEGER
22                   | REAL
23                   | BOOLEAN
24                   | CHAR
25
26 period            → range
27                   | period ',' range
28
29 range             → NUMBER DOTDOT NUMBER

```

8.5 变量声明

```

01 var_declarations → ε
02                 | VAR var_declaration_list ';'
03
04 var_declaration_list → var_declaration
05                     | var_declaration_list ';' var_declaration
06
07 var_declaration → idlist ':' type

```

8.6 子程序声明

```

01 subprogram_declarations → ε
02                         | subprogram_declarations subprogram ';'
03
04 subprogram              → PROCEDURE IDENTIFIER formal_parameter ';' subprogram_body
05                         | FUNCTION IDENTIFIER formal_parameter ':' basic_type ';'
subprogram_body

```

```

06
07 formal_parameter  → ε
08                   | '(' ')'
09                   | '(' parameter_list ')'
10
11 parameter_list    → parameter
12                   | parameter_list ';' parameter
13
14 parameter         → var_parameter
15                   | value_parameter
16
17 var_parameter     → VAR value_parameter
18
19 value_parameter   → idlist ':' type
20
21 subprogram_body   → const_declarations var_declarations compound_statement

```

8.7 语句

```

01 compound_statement → KW_BEGIN statement_list_opt KW_END
02
03 statement_list_opt → ε
04                   | ';' statement_list_opt
05                   | statement_list
06
07 statement_list     → nonempty_statement
08                   | statement_list ';' statement
09                   | statement_list ';' error           // 恐慌模式恢复
10
11 statement          → ε
12                   | nonempty_statement
13
14 nonempty_statement → variable ASSIGN expression
15                   | procedure_call
16                   | compound_statement
17                   | IF expression THEN statement ELSE statement
18                   | IF expression THEN statement      // %prec LOWER_THAN_ELSE
19                   | FOR IDENTIFIER ASSIGN expression TO expression DO statement
20                   | FOR IDENTIFIER ASSIGN expression DOWNT0 expression DO
statement
21                   | WHILE expression DO statement
22                   | BREAK
23                   | READ '(' variable_list ')'
24                   | WRITE '(' expression_list ')'

```

8.8 变量与过程调用


```

01 variable      → IDENTIFIER id_varpart
02
03 id_varpart    → ε
04               | id_varpart '[' expression_list ']'
05               | id_varpart '.' IDENTIFIER
06
07 procedure_call → IDENTIFIER
08               | IDENTIFIER '(' ')'
09               | IDENTIFIER '(' expression_list ')'

```

8.9 表达式

```

01 expression    → simple_expression
02               | simple_expression '=' simple_expression
03               | simple_expression NE simple_expression
04               | simple_expression '<' simple_expression
05               | simple_expression LE simple_expression
06               | simple_expression '>' simple_expression
07               | simple_expression GE simple_expression
08
09 simple_expression → term
10               | simple_expression '+' term
11               | simple_expression '-' term
12               | simple_expression OR term
13
14 term           → factor
15               | term '*' factor
16               | term '/' factor
17               | term DIV factor
18               | term MOD factor
19               | term AND factor
20
21 factor          → NUMBER
22               | CHARACTER
23               | STRING
24               | TRUE
25               | FALSE
26               | variable
27               | '(' expression ')'
28               | IDENTIFIER '(' ')'
29               | IDENTIFIER '(' expression_list ')'
30               | NOT factor
31               | '-' factor // %prec UMINUS
32               | '+' factor // %prec UPLUS

```

8.10 说明

1. ϵ 表示空产生式。

- 标识符 (IDENTIFIER) 不区分大小写, 词法阶段统一转为小写后返回。
- 关键字 KW_BEGIN/KW_END 分别对应 begin/end, 使用带前缀的 Token 名以避免与 C++ 关键字冲突。
- 注释支持嵌套: { ... } 形式, 通过 Flex Start Condition 实现。
- 运算符优先级 (从低到高): OR → AND → 关系运算符 → + - → * / DIV MOD → NOT → 一元 -/+。赋值 ASSIGN 为右结合且优先级高于各表达式运算符。
- 悬挂 else 通过 %prec LOWER_THAN_ELSE 解决, else 与最近的 if 绑定。
- 数组类型支持多维声明 (如 array [1..3, 2..4] of integer), 构建为嵌套 ArrayTypeNode。
- Record 扩展支持: type T = record ... end; 类型定义、字段访问 r.f、record 作为参数、record 数组。
- statement_list ';' error 为恐慌模式恢复规则: 语法分析器在分号后遇到错误时丢弃后续 Token 直到下一个分号, 然后恢复分析。

9. 附录 B: 测试报告

此处附上简要版的测试报告, 对于各模块的单元测试报告以及集成测试报告, 请见随报告提交的单独文档。

9.1 词法分析测试

9.1.1 合法用例: 关键字大小写不敏感

测试用例 v02_keyword_mixedcase.pas 验证词法分析器对 Pascal 关键字不区分大小写的处理能力。

测试源码:

```
01 ProGram t02;  
02 begin  
03 end.
```

编译命令:

```
01 ./build/pascc -i test/cases/valid/v02_keyword_mixedcase.pas --lex
```

Token 输出:

```
01 Type, Lexeme, Line, Column  
02 Keyword, ProGram, 1, 1  
03 Identifier, t02, 1, 9
```

```
04 Delimiter, ;, 1, 12
05 Keyword, begin, 2, 1
06 Keyword, end, 3, 1
07 Delimiter, ., 3, 4
```

词法分析器正确将大小写混合的 token 识别为 program、begin、end 关键字，t02 为标准标识符，行列号精确。

9.1.2 非法用例：未终止字符字面量

测试用例 i05_unterminated_char_literal.pas 验证词法分析器对未闭合字符字面量的错误检测。

测试源码：

```
01 program i05;
02 var c: char;
03 begin
04   c := 'a; { 未终止的字符字面量 }
05 end.
```

编译命令：

```
01 ./build/pascc -i test/cases/invalid/i05_unterminated_char_literal.pas --lex
```

错误输出：

```
01 Error at 4:8 - Lexical error: unterminated character literal (lexeme='a;')
```

词法分析器在编译阶段正确报告了未终止字符字面量错误，同时给出了精确的行号（第 4 行）和列号（第 8 列），以及引发错误的词素内容。

9.2 语法分析测试

9.2.1 合法用例：for-to 循环的 AST 构建

测试用例 `pv05_for_to.pas` 验证语法分析器对 `for-to` 循环的正确解析和 AST 构建。

测试源码：

```
01 program pv05;  
02 var i: integer;  
03 begin  
04     for i := 1 to 10 do  
05         i := i + 1  
06 end.
```

编译命令：

```
01 ./build/pascc -i test/cases/parser_valid/pv05_for_to.pas --parse
```

AST 输出：

```
01 Parse succeeded.  
02 Program  
03   Block  
04     List  
05     List  
06     List  
07       VarDecl  
08         List  
09           Identifier (i)  
10           Identifier (integer)  
11     List  
12     CompoundStmt  
13       ForStmt  
14         Identifier (i)  
15         Literal  
16         Literal  
17         AssignStmt  
18           Identifier (i)  
19           BinaryExpr  
20             Identifier (i)  
21             Literal
```

语法分析器生成了正确的 AST：ForStmt 节点包含循环变量 `i`、起始值 `1`、终止值 `10` 和循环体（赋值语句），各节点类型与层级结构均正确。

9.2.2 非法用例：if 缺少 then

测试用例 `pi04_if_missing_then.pas` 验证语法分析器对缺失关键字的错误检测与恐慌模式恢复。

测试源码:

```
01 program pi04;  
02 var a: integer;  
03 begin  
04     if a = 1  
05         a := 2  
06 end.
```

编译命令:

```
01 ./build/pascc -i test/cases/parser_invalid/pi04_if_missing_then.pas --parse
```

错误输出:

```
01 Parse error at 5:5 near 'a': syntax error  
02 Parsing failed.
```

语法分析器检测到 `if` 语句中缺少必需的 `then` 关键字，在第 5 行第 5 列报错并终止分析。

9.3 语义分析测试

9.3.1 合法用例：阶乘递归函数

测试用例 `sv17_func_result.pas` 验证语义分析器对递归函数声明与函数返回值赋值的处理。

测试源码:

```
01 program sv17_func_result;  
02 var n, result: integer;  
03  
04 function factorial(x: integer): integer;  
05 begin  
06     if x <= 1 then  
07         factorial := 1  
08     else  
09         factorial := x * factorial(x - 1)  
10 end;  
11  
12 begin  
13     n := 5;  
14     result := factorial(n)  
15 end.
```

编译命令:

```
01 ./build/pascc -i test/cases/semantic_valid/sv17_func_result.pas --semantic
```

语义分析输出:

```
01 Semantic analysis succeeded.
```

语义分析器正确识别了函数体内对函数名 `factorial` 的赋值为合法的函数返回值设置操作。

9.3.2 合法用例：多维数组

测试用例 `sv26_array_multidim.pas` 验证语义分析器对多维数组声明与下标访问的处理。

测试源码:

```
01 program sv26_array_multidim;
02 var
03     matrix: array[1..3, 1..3] of integer;
04     i, j, sum: integer;
05 begin
06     for i := 1 to 3 do
07         for j := 1 to 3 do
08             matrix[i, j] := i * j;
09     sum := 0;
10     for i := 1 to 3 do
11         sum := sum + matrix[i, i]
12 end.
```

语义分析输出:

```
01 Semantic analysis succeeded.
```

9.3.3 非法用例：类型不匹配赋值

测试用例 `si06_type_mismatch_assign.pas` 验证语义分析器对布尔值赋值给整数变量的类型错误检测。

测试源码:

```
01 program si06_type_mismatch_assign;
02 var a: integer;
03 begin
04     a := true
05 end.
```

编译命令:

```
01 ./build/pascc -i test/cases/semantic_invalid/si06_type_mismatch_assign.pas --  
semantic
```

错误输出：

```
01 Error at 5:5 - Type mismatch in assignment: expected integer, got boolean
```

9.3.4 非法用例：数组下标越界

测试用例 `si12_array_index_bounds.pas` 验证语义分析器对编译期常量下标越界的检测能力。

测试源码：

```
01 program si12_array_index_bounds;  
02 var arr: array[1..10] of integer;  
03 begin  
04     arr[0] := 1  
05 end.
```

编译命令：

```
01 ./build/pascc -i test/cases/semantic_invalid/si12_array_index_bounds.pas --  
semantic
```

错误输出：

```
01 Error at 5:9 - Array index out of bounds: 0 not in [1, 10]
```

语义分析器通过编译期常量求值，检测到下标 0 超出数组 `arr[1..10]` 的声明范围。

9.4 Record 专项测试

9.4.1 合法用例：record 数组求总年龄

测试用例 test_record_array_sum.pas 验证 record 数组的声明、字段访问、循环遍历与聚合运算。

测试源码：

```
01 program test;
02 type
03     person = record
04         age: integer;
05         score: real
06     end;
07 var
08     people: array[1..3] of person;
09     i: integer;
10     totalAge: integer;
11     avgScore: real;
12
13 begin
14     people[1].age := 20;
15     people[1].score := 80.0;
16     people[2].age := 25;
17     people[2].score := 90.0;
18     people[3].age := 30;
19     people[3].score := 100.0;
20
21     totalAge := 0;
22     avgScore := 0.0;
23
24     for i := 1 to 3 do
25     begin
26         totalAge := totalAge + people[i].age;
27         avgScore := avgScore + people[i].score
28     end;
29
30     avgScore := avgScore / 3.0;
31
32     write(totalAge)
33 end.
```

pascc 生成的目标 C 代码：

```
01 #include <stdio.h>
02 typedef struct {
03     int age;
04     float score;
```



```

05 } person;
06 person people[3];
07
08 int i;
09 int totalage;
10 float avgscore;
11
12 int main(void) {
13     people[(1) - 1].age = 20;
14     people[(1) - 1].score = 80.0;
15     people[(2) - 1].age = 25;
16     people[(2) - 1].score = 90.0;
17     people[(3) - 1].age = 30;
18     people[(3) - 1].score = 100.0;
19     totalage = 0;
20     avgscore = 0.0;
21     for (i = 1; i <= 3; i++) {
22         totalage = (totalage + people[(i) - 1].age);
23         avgscore = (avgscore + people[(i) - 1].score);
24     }
25     avgscore = (avgscore / 3.0);
26     printf("%d", totalage);
27     return 0;
28 }

```

C 编译与运行:

```

01 $ gcc -std=c99 test_record_array_sum.c -o test_record_array_sum
02 $ ./test_record_array_sum
03 75

```

编译器正确实现了 Pascal record 类型到 C struct 的映射、array[1..3] 到 [3] 的下标偏移 ((i) - 1)、以及字段访问 people[i].age 的忠实翻译。程序输出 75 (20 + 25 + 30)，验证正确。

9.4.2 合法用例：record 作为 var 参数

测试用例 test_record_param_var.pas 验证 record 类型作为 var 引用参数传入过程的正确性——参数修改能反映回调用方。

测试源码:

```

01 program test;
02 type
03     person = record
04         age: integer;
05         score: real
06     end;

```

```

07 var p: person;
08
09 procedure modifyPerson(var p: person);
10 begin
11     p.age := 30;
12     p.score := 88.0
13 end;
14
15 begin
16     p.age := 25;
17     p.score := 95.5;
18     write(p.age);
19     modifyPerson(p);
20     write(p.age)
21 end.

```

pascc 生成的目标 C 代码:

```

01 #include <stdio.h>
02 typedef struct {
03     int age;
04     float score;
05 } person;
06 person p;
07
08 void modifyperson(person *p);
09
10 void modifyperson(person *p) {
11     (*p).age = 30;
12     (*p).score = 88.0;
13 }
14
15 int main(void) {
16     p.age = 25;
17     p.score = 95.5;
18     printf("%d", p.age);
19     modifyperson(&p);
20     printf("%d", p.age);
21     return 0;
22 }

```

C 编译与运行:

```

01 $ gcc -std=c99 test_record_param_var.c -o test_record_param_var
02 $ ./test_record_param_var
03 2530

```

关键观察:

- Pascal var 参数正确翻译为 C 指针参数 (person *p)。

- 过程体内对 `record` 字段的访问编译为 C 指针解引用 `(*p).age`。
- 调用方自动传递变量地址 `&p`。
- 输出 2530: 修改前 `p.age = 25` (输出 25), `modifyPerson` 修改后 `p.age = 30` (输出 30), 验证引用传递正确。

9.4.3 非法用例：访问不存在的 `record` 字段

测试用例 `test_record_invalid_field.pas` 验证语义分析器对 `record` 字段访问的静态检查。

测试源码：

```
01 program test;
02 type
03     person = record
04         age: integer;
05         score: real
06     end;
07 var p: person;
08
09 begin
10     p.name := 'John'; { Error: field 'name' does not exist }
11     write(p.age)
12 end.
```

语义分析输出：

```
01 Error at 11:3 - Record type 'person' has no field 'name'
02 Error at 11:3 - Type mismatch in assignment: expected unknown, got char
```

9.5 课件示例测试

课件中示例程序的扩展，验证编译器对完整标准 Pascal-S 程序的端到端编译能力。

测试源码 `test/cases/official/1.pas`:

```
01 program example(input, output);
02 const t = 's'; a = 1e6;
03 var x, y: integer;
04     z : array [1..10, 2..8] of integer;
05     u : integer;
06
07 function gcd(a, b: integer): integer;
08 begin
09     if b=0 then gcd:=a
10     else gcd:=gcd(b, a mod b)
11 end;
12
13 begin
14     x := 2+1;
15     z[2,2] := 5;
16     u := 1;
17     read(x, y);
18     write(gcd(x, y))
19 end.
```

该程序定义了 Pascal-S 的多种语言特性：常量声明（字符常量 's'、科学计数法实数 1e6）、多维数组声明（array[1..10, 2..8]）、递归 GCD 函数、以及输入输出语句。编译器能够完整编译该程序，生成正确可编译的 C 代码，通过 gcc 无警告编译，运行时正确计算两数的最大公约数并输出。

9.6 Open Set 集成测试

9.6.1 扩展欧几里得算法

测试用例 43_exgcd.pas 验证编译器对递归函数、var 参数、整数除法和取模运算的完整编译。

测试源码:

```
01 program main;
02 var
03     a, b: integer;
04
05 function exgcd(a, b: integer; var x, y: integer): integer;
06 var t, r: integer;
```

```

07 begin
08   if b = 0 then
09     begin
10       x := 1;
11       y := 0;
12       exgcd := a;
13     end
14   else
15     begin
16       r := exgcd(b, a mod b, x, y);
17       t := x;
18       x := y;
19       y := (t - (a div b) * y);
20       exgcd := r;
21     end;
22 end;
23
24 begin
25   a := 7;
26   b := 15;
27   exgcd(a, b, x[0], y[0]);
28   x[0] := ((x[0] mod b) + b) mod b;
29   write(x[0]);
30 end.

```

pascc 生成的目标 C 代码:

```

01 #include <stdio.h>
02 int a; int b;
03
04 int exgcd(int a, int b, int *x, int *y);
05
06 int exgcd(int a, int b, int *x, int *y) {
07     int _retval; int t; int r;
08     if ((b == 0)) {
09         (*x) = 1;
10         (*y) = 0;
11         _retval = a;
12     } else {
13         r = exgcd(b, (a % b), x, y);
14         t = (*x);
15         (*x) = (*y);
16         (*y) = (t - ((a / b) * (*y)));
17         _retval = r;
18     }
19     return _retval;
20 }
21
22 int main(void) {

```

```

23     a = 7; b = 15;
24     exgcd(a, b, &x[0], &y[0]);
25     x[0] = (((x[0] % b) + b) % b);
26     printf("%d", x[0]);
27     return 0;
28 }

```

C 编译与运行 (gcc 产生 1 个无关警告):

```

01 $ gcc -std=c99 exgcd.c -o exgcd
02 $ ./exgcd
03 13

```

程序使用扩展欧几里得算法求解 7 关于模数 15 的乘法逆元，结果为 13 ($7 \times 13 \equiv 1 \pmod{15}$)，验证编译正确。

9.6.2 图着色计数 (DP)

测试用例 42_color.pas 是一个较复杂的动态规划程序，求解图的合法着色方案数，对编译器多维数组访问、常量表达式和模运算的处理形成压力测试。

核心递归函数源码片段 (完整程序 55 行):

```

01 function dfs(a, b, c, d, e, last: integer): integer;
02 var anss: integer;
03 begin
04     if dp[a, b, c, d, e, last] <> -1 then
05         dfs := dp[a, b, c, d, e, last];
06     if a + b + c + d + e = 0 then
07         dfs := 1
08     else
09         begin
10             anss := 0;
11             if a <> 0 then
12                 anss := (anss + (a - equal(last, 2))
13                     * dfs(a - 1, b, c, d, e, 1)) mod modn;
14             { ... 其余四个分支类似 ... }
15             dp[a, b, c, d, e, last] := anss mod modn;
16             dfs := dp[a, b, c, d, e, last];
17         end;
18 end;

```

该程序使用了五维 DP 数组 `dp[0..17, 0..17, 0..17, 0..17, 0..17, 0..6]`，六层嵌套 for 循环进行初始化，以及递归 DFS 中的多维数组访问和模运算。

pascc 生成的多维数组下标偏移 (C 代码片段):

```

01 int dp[18][18][18][18][18][7];
02 // ...
03 cns[(list[i]) - 1] = (cns[(list[i]) - 1] + 1);
04 ans = dfs(cns[(1) - 1], cns[(2) - 1], cns[(3) - 1],
05           cns[(4) - 1], cns[(5) - 1], 0);

```

C 编译与运行:

```

01 $ gcc -std=c99 color.c -o color
02 $ ./color < 42_color.in
03 39480

```

即便存在 gcc 的括号风格警告，程序编译后正确运行并输出期望结果 39480，验证了编译器在复杂算法场景下的多维数组翻译、var 参数处理和模运算的正确性。

10. 附录 C: Git 日志

```

01 commit a2949d8de4fd675ca6f2a2300c3e066961a95bcd
02 Author: 张宸宇 <hxd3c0015@126.com>
03 Date: Sat May 16 23:01:46 2026 +0800
04
05 docs: v1.2
06
07 commit 8b3c581cdd80940549eb7b13a2d00ceblac7ff86
08 Author: 张宸宇 <hxd3c0015@126.com>
09 Date: Mon May 11 21:45:51 2026 +0800
10
11 docs: 词法分析报告完善, main-v1.1.typ
12
13 commit e574c35ec30f4059d2a2c9344e44911b2e3827f7
14 Merge: 2027faf 9b79db9
15 Author: Zhang Chenyu <141384271+invalidval@users.noreply.github.com>
16 Date: Mon May 11 19:52:49 2026 +0800
17
18 Merge pull request #10 from invalidval/feature/record
19
20 docs: 课程报告初版
21
22 commit 9b79db957e2ac99d9f654546a14801f5f483f7e2
23 Author: 张宸宇 <hxd3c0015@126.com>
24 Date: Mon May 11 19:47:03 2026 +0800
25
26 docs: 课程报告初版
27
28 commit 2027faf1540d6c7ec14146f3fede63c7a7afda5b
29 Merge: e53656a 890e825

```

```
30 Author: d& <141384271+invalidval@users.noreply.github.com>
31 Date: Fri Apr 24 16:59:59 2026 +0800
32
33 Merge pull request #9 from invalidval/feature/record
34
35 feat&docs: record array/param
36
37 commit 890e825f4321f3f17d14dbf56b1b5eab0e2f7647
38 Author: 张宸宇 <hxd3c0015@126.com>
39 Date: Fri Apr 24 16:57:46 2026 +0800
40
41 feat&docs: record array/param
42
43 commit e53656a544bcf1157c030280e71765302ee743a2
44 Merge: f6cd49b 7f041b7
45 Author: d& <141384271+invalidval@users.noreply.github.com>
46 Date: Thu Apr 23 13:46:05 2026 +0800
47
48 Merge pull request #8 from invalidval/feature/record
49
50 Feature/record
51 增加record
52 增加IDE喵^^
53
54 commit 7f041b71132fc3704c5a505e1aa57c539d52d310
55 Author: 张宸宇 <hxd3c0015@126.com>
56 Date: Thu Apr 23 13:39:33 2026 +0800
57
58 feat: IDE
59
60 commit ddf4c41ead7cc47280630f8a223f0df2804d8fcc
61 Author: 张宸宇 <hxd3c0015@126.com>
62 Date: Thu Apr 23 12:40:45 2026 +0800
63
64 docs: record-type
65
66 commit 06557a440c43c9f01ab892faf6c3ce115b133eb5
67 Author: 张宸宇 <hxd3c0015@126.com>
68 Date: Thu Apr 23 10:13:42 2026 +0800
69
70 feat: add record support
71
72 commit f6cd49bcd47ce6374538d699bee1cbaff9901f2
73 Author: 张宸宇 <hxd3c0015@126.com>
74 Date: Wed Apr 22 20:42:21 2026 +0800
75
76 chore: 移除 .DS_Store 文件
77
78 commit e452a281220c4a97e3c2f54f58d8766fba890f37
```



```
79 Author: 张宸宇 <hxd3c0015@126.com>
80 Date: Wed Apr 22 19:10:27 2026 +0800
81
82 feat: better tui
83
84 commit 8499ee5537c1d643b745e23092ed9e8ed7a272aa
85 Author: 张宸宇 <hxd3c0015@126.com>
86 Date: Wed Apr 22 13:40:24 2026 +0800
87
88 feat: tui
89
90 commit 306bfa239b3b05e7edf5bb51bb7e206f363a4c5b
91 Author: 胡航宾 (Hangbin Hu) <142589330+Brcoinlearning@users.noreply.github.com>
92 Date: Tue Apr 21 22:34:21 2026 +0800
93
94 docs: add lexer midterm slides and figures
95
96 commit blfed8682f2706243918468c944a2ff1d19ff58d
97 Author: 张宸宇 <hxd3c0015@126.com>
98 Date: Tue Apr 21 20:53:12 2026 +0800
99
100 ppt: merged wjh and zcy works
101
102 commit clae8dee46d1984b8b10b6082d8223a3501224ae
103 Author: 张宸宇 <hxd3c0015@126.com>
104 Date: Tue Apr 21 10:41:55 2026 +0800
105
106 合并了一个ppt, 并回退不正确merge
107
108 commit 73055af467ba6820d22b10e29140bd26226e32ab
109 Author: 19daze <3120623907@qq.com>
110 Date: Sat Apr 18 21:30:13 2026 +0800
111
112 week8 中期汇报-语法部分; 修复了归约-归约冲突
113
114 commit 2022826d3cfe5446d5e60c82ef9f9ce8d0c55dda
115 Author: 张宸宇 <hxd3c0015@126.com>
116 Date: Thu Apr 16 20:25:20 2026 +0800
117
118 docs: ppt init
119
120 commit e7f4bb7800a7debb6805defdd5896fa4aaa9e3d4
121 Author: 张宸宇 <hxd3c0015@126.com>
122 Date: Sat Apr 11 01:09:12 2026 +0800
123
124 refactor: c style
125
126 commit 19be346d3d83f6f384a81b91a3991bdeb42e2e18
127 Author: 张宸宇 <hxd3c0015@126.com>
```

```

128 Date: Sat Apr 11 00:55:38 2026 +0800
129
130 refactor: &(*x)
131
132 commit 718070333a40df20bc84cd64f0ef9ebf901bed01
133 Author: 张宸宇 <hxd3c0015@126.com>
134 Date: Fri Apr 10 23:30:08 2026 +0800
135
136 refactor: something
137
138 commit 209eddcdd6ac8fce22825f03e1f0c2ebf8df8807
139 Author: 张宸宇 <hxd3c0015@126.com>
140 Date: Fri Apr 10 22:59:05 2026 +0800
141
142 docs: 修复日志
143
144 commit 8f0e9a1e03dd38568808603d666053cc5216653a
145 Author: 张宸宇 <hxd3c0015@126.com>
146 Date: Fri Apr 10 22:57:11 2026 +0800
147
148 finish: 100passed
149
150 commit bea974c1d4642664dd0268f59d0289fc1c852de2
151 Author: 张宸宇 <hxd3c0015@126.com>
152 Date: Fri Apr 3 14:52:03 2026 +0800
153
154 docs:详细设计
155
156 commit d511ad8c02bfb0349912c6ae72ed1ed5b928988d
157 Merge: 290aaa0 a7ff689
158 Author: d& <141384271+invalidval@users.noreply.github.com>
159 Date: Wed Apr 1 11:20:18 2026 +0800
160
161 Merge pull request #7 from invalidval/语法改错v2
162
163 语法改错v2
164
165 commit a7ff689494361d34a23c007a6d00b782757e1be4
166 Merge: 3c1938e ad26013
167 Author: 张宸宇 <hxd3c0015@126.com>
168 Date: Wed Apr 1 11:17:12 2026 +0800
169
170 Merge branch 'edited_by_lsy' into 语法改错v2
171
172 修复语法、词法
173 合并文档
174
175 commit 3c1938e889448ab9368c482d73c2e9e243663760
176 Author: 张宸宇 <hxd3c0015@126.com>

```

177 Date: Wed Apr 1 11:16:42 2026 +0800
178
179 增加break关键字
180
181 commit ad2601338d1025b2ab1026ec325a391ddc9cbbce
182 Author: 张宸宇 <hxd3c0015@126.com>
183 Date: Wed Apr 1 10:46:30 2026 +0800
184
185 docs
186
187 commit e05e900731fca929c0f54dbd9e07f44b6eb787f5
188 Author: 19daze <3120623907@qq.com>
189 Date: Wed Apr 1 10:08:28 2026 +0800
190
191 语法改错v2：单引号多字符识别支持，数组下标支持a[i][j]写法
192
193 commit 290aaa003557de42b64cad2fc74cd97d9762cd78
194 Merge: ee6bda7 87c4ab8
195 Author: d& <141384271+invalidval@users.noreply.github.com>
196 Date: Mon Mar 30 17:30:45 2026 +0800
197
198 Merge pull request #6 from invalidval/edited_by_lsy
199
200 Edited by lsy
201
202 commit 87c4ab8d1548b728e0568d56ae23d782c2f39c4b
203 Author: 张宸宇 <hxd3c0015@126.com>
204 Date: Mon Mar 30 17:29:54 2026 +0800
205
206 修复语义检查对 read 的误报
207
208 commit 78f4aec7ea6420d5d048e13dcd4aba212e069bd0
209 Author: 张宸宇 <hxd3c0015@126.com>
210 Date: Mon Mar 30 17:05:05 2026 +0800
211
212 增加用例，修改文档图标
213
214 commit ee6bda787a869b06cf419f0cfd7302c73702ee0f
215 Merge: 700951c 5f3b5e1
216 Author: d& <141384271+invalidval@users.noreply.github.com>
217 Date: Sun Mar 29 15:03:28 2026 +0800
218
219 Merge pull request #5 from invalidval/edited_by_lsy
220
221 Edited by lsy
222
223 commit 5f3b5e1a411fbd8c6ac803dc7a6107a8c3946916
224 Author: 张宸宇 <hxd3c0015@126.com>
225 Date: Sun Mar 29 15:01:12 2026 +0800

226
227 修复代码生成变量**类型回退为int, 不按AST标注来**的问题
228
229 commit 24c085849d29fa000ecb311cb27855b74b67270c
230 Author: 张宸宇 <hxd3c0015@126.com>
231 Date: Sat Mar 28 23:01:38 2026 +0800
232
233 依旧修复大量bug
234
235 commit 99959edab01407ceff80a153444929fe39d9bd
236 Author: lsiyuan <lsiyuan20050224@gmail.com>
237 Date: Sat Mar 28 20:47:08 2026 +0800
238
239 v0提交
240
241 commit 9ca69b13c91fcc9a91830913b5081e58e324aa
242 Author: 19daze <3120623907@qq.com>
243 Date: Sat Mar 28 17:03:13 2026 +0800
244
245 改正了语法错误
246
247 commit 700951c1fa26b6529164894f480833062c01b44c
248 Merge: 2035393 3e6813c
249 Author: d& <141384271+invalidval@users.noreply.github.com>
250 Date: Fri Mar 27 10:06:48 2026 +0800
251
252 Merge pull request #4 from invalidval/semantic
253
254 支持非运算
255 - 语义分析错误处理完成
256
257 commit 3e6813c8bbeaabbf177e57080ab97c94727420caa
258 Author: 张宸宇 <hxd3c0015@126.com>
259 Date: Thu Mar 26 23:18:16 2026 +0800
260
261 支持非运算
262
263 commit 2035393dfddc7970d99a67ad2bfbdd6aeb80ad2c
264 Merge: 30300f8 74b2a90
265 Author: d& <141384271+invalidval@users.noreply.github.com>
266 Date: Thu Mar 26 20:54:38 2026 +0800
267
268 Merge pull request #3 from invalidval/代码生成
269
270 代码生成
271 bug多
272
273 commit 74b2a9085e2bf81c1333ce17d23360a12db2415b
274 Author: 张宸宇 <hxd3c0015@126.com>

```

275 Date: Thu Mar 26 20:52:33 2026 +0800
276
277 支持boolean, 增加自动化脚本
278
279 commit c78ff9f836108f28d8589e71dd72e8c65a70731a
280 Author: 张宸宇 <hxd3c0015@126.com>
281 Date: Thu Mar 26 20:18:59 2026 +0800
282
283 测试集open_set
284
285 commit e273d2999c897e4ae1da42234448e274b0be9c46
286 Author: 张宸宇 <hxd3c0015@126.com>
287 Date: Thu Mar 26 17:49:09 2026 +0800
288
289 修复大量代码生成bug, 可基本运行
290
291 commit 13e4a5a42ef319de9be8853837f6ffb1fcd8497f
292 Author: sanbao0220 <2023211177@bupt.cn>
293 Date: Thu Mar 26 16:34:46 2026 +0800
294
295 解决构建时报错问题
296
297 commit 80bab588de96d5997f7ec7ffa237b7b5a346fd34
298 Author: sanbao0220 <2023211177@bupt.cn>
299 Date: Thu Mar 26 15:37:23 2026 +0800
300
301 完成代码实现,待测试
302
303 commit 30300f80e58a6a7b04cabef69c6360f9d229abe6
304 Merge: 0daedc7 cd6253e
305 Author: d& <141384271+invalidval@users.noreply.github.com>
306 Date: Tue Mar 24 17:20:02 2026 +0800
307
308 Merge pull request #2 from invalidval/semantic
309
310 Semantic
311 - 支持Int类型向Real兼容
312 - 完善语义分析补充说明
313 - 修正语义分析测试用例
314 - 增加main参数
315
316 commit cd6253e36ad7f2340642de0f72a36e1e8a8b0e0e
317 Author: 张宸宇 <hxd3c0015@126.com>
318 Date: Tue Mar 24 17:17:17 2026 +0800
319
320 支持Int向Real兼容; 补充文档
321
322 commit 98f643f40f872315318ef5ed3d9210e703c20c17
323 Author: 张宸宇 <hxd3c0015@126.com>

```

```
324 Date: Tue Mar 24 14:37:32 2026 +0800
325
326 修正semantic参数、测试用例
327
328 commit 0daedc7d03e53a2f77f044481aca3d2b866faf48
329 Merge: 80b6e44 3d36182
330 Author: d& <141384271+invalidval@users.noreply.github.com>
331 Date: Mon Mar 23 14:58:23 2026 +0800
332
333 Merge pull request #1 from invalidval/semantic
334
335 Semantic v0.1
336
337 commit 3d36182e2d17855399469e17d18705c70affbd57
338 Author: 张宸宇 <hxd3c0015@126.com>
339 Date: Mon Mar 23 14:42:39 2026 +0800
340
341 feature: semantic v0.1
342
343 commit d3f9bca5c534608fc79673c04276670427c805d1
344 Author: zcy <zcy@appledeMacBook-Pro.local>
345 Date: Sun Mar 22 20:45:30 2026 +0800
346
347 合并语法分析(待联调)
348
349 commit ec07270b25b9f1dc381a5f0b7ff55b23ec804a23
350 Author: 19daze <3120623907@qq.com>
351 Date: Sun Mar 22 19:36:39 2026 +0800
352
353 完成了语法分析实现和仅语法环境下的测试跑通
354
355 commit a244e67ec8de54a79f5081e075ce4780e6ff132c
356 Author: 19daze <3120623907@qq.com>
357 Date: Sat Mar 21 22:26:29 2026 +0800
358
359 AST契约相关内容
360
361 commit 80b6e44e6e9c0f3f0da00e79708762e51c19db52
362 Author: zcy <zcy@appledeMacBook-Pro.local>
363 Date: Sat Mar 21 22:17:54 2026 +0800
364
365 合并词法分析，并迁移了个人文档位置
366
367 commit 9ee2bc49c582049976efbfa861d6f1ad615a2a9c
368 Author: 胡航宾 (Hangbin Hu) <142589330+Brcoinlearning@users.noreply.github.com>
369 Date: Sat Mar 21 20:26:19 2026 +0800
370
371 docs(lexer): 修缮词法契约与测试验收文档
372
```

```

373 commit 217721efc454076ba4d8efca815d2514d264c9e0
374 Author: 胡航宾 (Hangbin Hu) <142589330+Brcoinlearning@users.noreply.github.com>
375 Date: Sat Mar 21 19:37:32 2026 +0800
376
377     feat(lexer): 完成词法分析实现与测试资产
378
379 commit fa7f885fc72841aef9f86c133e6ba10337e5a9a4
380 Author: zcy <zcy@appledeMacBook-Pro.local>
381 Date: Fri Mar 20 16:41:39 2026 +0800
382
383     更新了文档
384
385 commit 38ede69a1db1745f758c9ff6c08e4bbecc750d02
386 Author: zcy <zcy@appledeMacBook-Pro.local>
387 Date: Thu Mar 19 15:30:08 2026 +0800
388
389     Remove build/ and output/ from version control
390
391 commit 4c9519e5cd27bae5ee19a706bd66b025e9378809
392 Author: zcy <zcy@appledeMacBook-Pro.local>
393 Date: Thu Mar 19 11:02:44 2026 +0800
394
395     ignore build
396
397 commit 5bac3b737f6e53828c02a4d33adceeb3feb675cf
398 Author: zcy <zcy@appledeMacBook-Pro.local>
399 Date: Thu Mar 19 11:00:16 2026 +0800
400
401     Add docs, ast
402
403 commit 4c8e70e342b853545c7e23fd4700144d18a32463
404 Author: zcy <zcy@appledeMacBook-Pro.local>
405 Date: Wed Mar 18 16:05:31 2026 +0800
406
407     Skeleton
408
409 commit 4a5c96daa1aa7dcbfeef599ed013d7c873de646e
410 Author: d& <141384271+invalidval@users.noreply.github.com>
411 Date: Fri Mar 13 16:05:25 2026 +0800
412
413     Initial commit

```

11. 附录 D：大模型使用情况记录

在本次课设中，我们结合了大模型软件开发的技术。我们的角色从底层编码实现向顶层设计转移，但是并没有将所有要求丢给大模型后一劳永逸，而是经过仔细的规划、分析、设计后，带着问题和思考，用清晰、明确的 prompt 让

大模型根据我们的要求写出符合预期的代码。即使实现不符合预期，我们仍能凭借 C++ 和编译原理基础发现问题，并提出修改方案。

下面附上团队协作过程中的大模型使用记录飞书文档：